



Universidad
Internacional
de Valencia

Máster Universitario en Robótica y Automatización de Procesos Industriales

Sistemas Robóticos Móviles

Actividad 2

Simulación de celda warehouse con Pioneer, mannequin Bill, herramientas y SLAM en CoppeliaSim y Lua

Entrega práctica de Warehouse R1-B1-T1 y T2

Estudiante

Jorge Luis Mayorga Taborda

Profesor Jose I. Iñiguez

jiniguez@professor.universidadviu.com

Fecha 26 de mayo de 2026, antes de las 23:59

Índice general

1	Introducción	6
1.1	Vista general del resultado	7
2	Escenarios CoppeliaSim	9
2.1	Elementos del warehouse	9
2.2	Cómo funciona el Pioneer dentro de CoppeliaSim.....	9
2.3	Bill como actor móvil.....	10
2.4	Cola de tareas	10
2.5	Integración del código	11
2.6	Capturas de la simulación	13
2.7	Piezas visuales	13
3	Modelo del robot y control	14
3.1	Seguimiento de objetivo	14
3.2	Modos de control evaluados.....	15
3.2.1	Capas de moldeado de velocidad sobre el modo base	16
4	Seguimiento de Bill y comparación de controladores	18
4.1	Geometría de seguimiento	19
4.2	Modelo cinemático usado por todos los modos	19
4.3	P, PI y PID	20
4.4	LQR discreto.....	21
4.5	NMPC por disparo directo	22
4.6	Resultados de seguimiento	23
5	Campos potenciales, evitación de obstáculos y planificación local	26
5.1	Campo atractivo hacia Bill	26
5.2	Campo reactivo de obstáculos.....	26
5.3	Visualización de sensores.....	27
5.4	Punto intermedio de planificación SLAM.....	27
6	Estimación y SLAM	28
6.1	Sensores: LiDAR, ultrasonidos y rayos de proximidad	28
6.2	Datos usados para comparar SLAM	29
6.3	Por qué hace falta Kalman	30

6.4	Predicción de pose	30
6.5	Corrección Kalman-landmark	31
6.6	GMapping en grid	32
6.7	Hector con ajuste local	33
6.8	Cartographer con submapas	34
6.9	Qué método usar y por qué	35
7	Resultados experimentales	36
7.1	Validación funcional de la escena	36
7.2	Desempeño del estimador Kalman-landmark	37
7.3	Comparación de algoritmos SLAM	38
7.4	Interpretación de resultados	39
8	Fase 2: SLAM con mapa desconocido	40
8.0.1	¿Mejora el SLAM con cada vuelta?	41
8.1	Recuperación ante bloqueo local	43
8.2	Comparación de métodos	44
8.3	Discusión técnica	46
9	Validación	47
9.1	Validación por lista de verificación	47
9.2	Timeline de la misión	48
9.3	Incidencias corregidas	49
9.3.1	Sincronización de tarea T2	49
9.3.2	Ciclos del planificador en Fase 2	49
9.3.3	Oscilación del scan matching tipo Hector	50
	Conclusiones	51
	Referencias	53
A	Anexos	55
A.1	Archivos principales	55
A.2	Parámetros relevantes	57
A.3	Fragmentos Lua propios	57
A.4	Comparación de control	58
A.5	Parámetros de Cartographer	59
A.6	Notas de alcance	59

Índice de figuras

1.1	Vista superior de la celda base Sim_T2_Phase1_Basic.....	8
2.1	Máquina de estados resumida del controlador R1 para entrega, trabajo de Bill, retorno de herramienta y carga.....	11
2.2	Capturas 3D de la escena ejecutada en CoppeliaSim.	13
3.1	Comparación de modos de control exportada desde CoppeliaSim. Todos los modos completan la tarea. PID obtiene el menor puntaje ponderado con los pesos definidos ($n = 1$ por modo).	16
4.1	Escena de seguimiento: Bill define un objetivo móvil y R1 regula distancia, rumbo y suavidad de ruedas.....	18
4.2	Capturas de ejecución de Actividad2_1_Basic_Follower.ttt: Bill recorre una trayectoria circular y R1 mantiene una distancia de seguimiento estable.	18
4.3	Referencia móvil del seguidor: el robot controla un punto detrás de Bill y no el centro de la persona.	19
4.4	Comparación temporal de P, PI, PID, LQR y NMPC en la escena de seguimiento de Bill.	24
4.5	Coste físico aproximado del seguidor: potencia instantánea y energía acumulada en las ruedas.	24
4.6	Suavidad de comandos del seguidor: variación de las órdenes de rueda y cambios de sentido.....	25
6.1	Secuencia de estimación, mapa, planificación local y control de R1.....	29
6.2	Particularidad de Kalman-landmark: corrige la pose y fusiona pocas detecciones como rasgos; no mantiene una covarianza conjunta pose-mapa como un EKF-SLAM completo.	32
6.3	Particularidad de GMapping implementado: la información principal es una grid de ocupación. Cada rayo libera celdas antes del impacto y refuerza la celda ocupada.	32
6.4	Hector SLAM: iteración Gauss-Newton con interpolación bilineal del mapa log-odds. El Hessiano se acumula con los jacobianos analíticos de la transformación rígida; el sistema $H\Delta\xi = b$ se resuelve por la regla de Cramer.....	33
6.5	Particularidad de Cartographer-submap: el mapa no se guarda como una sola grid monolítica, sino como submapas locales con posibles cierres cuando R1 vuelve a zonas conocidas.	34
7.1	Resultados del estimador Kalman-landmark.....	37
7.2	Comparación gráfica de los cuatro modos SLAM: trayectoria, precisión, crecimiento del mapa y resumen de error.	39

8.1	Capturas 3D de la escena Fase 2 ejecutada en CoppeliaSim.....	42
8.2	Capturas de ejecución de Fase 2 con mapa desconocido y obstáculos en la celda.	43
8.3	Comparación de métodos en Fase 2.	45
8.4	Reconstrucción del mapa al mismo instante común, $t = 60$ s.	45
9.1	Secuencia temporal de la simulación: estados de tarea, movimiento, planificador y acciones de Bill durante la misión.	48
9.2	Panel de la ejecución base: pose, error, batería, obstáculos, planificador y estados principales.	49
A.1	Métricas complementarias de control: tiempos, trayectoria, suavidad, batería, riesgo y error de pose.	59

Índice de cuadros

1.1	Trazabilidad entre el enunciado de la Actividad 2 y la solución implementada.	7
2.1	Elementos de la celda de almacén simulada en CoppeliaSim.	9
4.1	Métricas del seguidor de Bill por modo de control.	23
6.1	Criterio de selección entre las variantes SLAM implementadas.	35
7.2	Comparación SLAM en CoppeliaSim.	38
8.1	Métricas de navegación por fase en la celda desconocida (media de 3 ejecuciones). “Explorar” es la pasada inicial de mapeo; las vueltas 1–3 son los ciclos de trabajo sobre el mapa acumulado.	41
8.2	Calidad del SLAM por fase en la celda desconocida (media de 3 ejecuciones): convergencia del mapa y precisión de localización.	41
8.3	Comparación Fase 2 con mapa desconocido y un robot móvil dinámico.	44
A.1	Escenas y controladores principales.	55
A.2	Exportadores, registros y escena complementaria.	56
A.3	Parámetros de configuración del sistema.	57
A.4	Resultados comparativos de los cinco controladores en la misión warehouse de Fase 1 ($n = 1$ por modo; puntaje compuesto, menor es mejor). Aquí LQR y NMPC son las variantes reactivas simplificadas del controlador de tarea; el LQR con ganancia de Riccati y el NMPC de horizonte completo se evalúan sobre el follower (Tabla 4.1).	58
A.5	Parámetros de Cartographer-submap. Idénticos en Fase 1 y Fase 2.	59

1

Introducción

Se implementó un Pioneer 3-DX en CoppeliaSim y Lua capaz de navegar autónomamente en una celda robotizada con evitación de obstáculos. La escena representa un caso de fábrica sencillo donde R1 asiste a B1, llamado Bill, una persona tomada del modelo disponible en CoppeliaSim y usada como *mannequin* operativo. Los fundamentos de robótica móvil diferencial y navegación autónoma se describen en [1]. El robot transporta piezas a dos mesas, espera a que Bill trabaje y devuelve cada herramienta a su estantería antes de volver a carga.

El ciclo básico coordina los tiempos con Bill, evita obstáculos locales, distingue una estantería de un bloqueo y devuelve cada herramienta antes de volver a carga.

Las cuatro variantes de estimación y mapeo (KALMAN_LANDMARK, GMAPPING_GRID, HECTOR_GRID_MATCHING y CARTOGRAPHER_SUBMAP) son desarrollos propios en Lua: el *núcleo* de cada método es genuino y se calcula en vivo a partir de los sensores, pero sin el componente de optimización pesado del original. La variante de GMapping mantiene una grid de ocupación *log-odds* (sin el filtro de partículas Rao-Blackwell); la de Hector resuelve el ajuste *scan-to-map* con un paso de Gauss-Newton real sobre la grid interpolada bilinealmente (a una sola resolución, sin la pirámide multi-resolución del original); la Kalman-landmark corrige la pose con un EKF de rango/rumbo y compuerta de Mahalanobis; y la de Cartographer mantiene submapas locales y detecta cierres de ciclo con una corrección de pose local (sin el grafo de poses global ni el optimizador Ceres). La comparación evalúa cuatro representaciones de mapa bajo las mismas condiciones de misión.

`actividad2/coppeliasim/Sim_T2_Phase1_Basic.ttt`

En este trabajo el *mannequin* se implementa como Bill/B1, el modelo de persona móvil disponible en CoppeliaSim. R1 sigue una referencia detrás de Bill, transporta T1 y T2, respeta esperas de trabajo humano, evita pallet, pilar, caja y obstáculos de Fase 2, y vuelve a la estación de carga. Se añade una capa de validación cuantitativa con métricas exportadas, comparación de controladores y mapeo local (Tabla 1.1).

Punto de la guía	Dónde queda cubierto
Seguir al <i>mannequin</i>	Bill/B1 actúa como objetivo móvil y como solicitante de herramientas en la celda.
Campos potenciales	Capítulo de campos potenciales: atracción hacia Bill y repulsión por sensores.
Anticolisión	Lectura de 16 sensores de proximidad, reducción de velocidad, giro reactivo y parada crítica.
Integración en celda	Warehouse con racks, mesas WS1/WS2, herramientas T1/T2, estación de carga y máquina de estados.
Ampliación del script base	Seguimiento P/PI/PID/LQR/NMPC, estimación Kalman, variantes SLAM y Fase 2 con mapa desconocido.

Cuadro 1.1: Trazabilidad entre el enunciado de la Actividad 2 y la solución implementada.

1.1 Vista general del resultado

La Fig. 1.1 muestra la vista superior de la celda base, con R1, B1, las estanterías de T1 y T2, las mesas de trabajo y los obstáculos presentes desde el inicio.

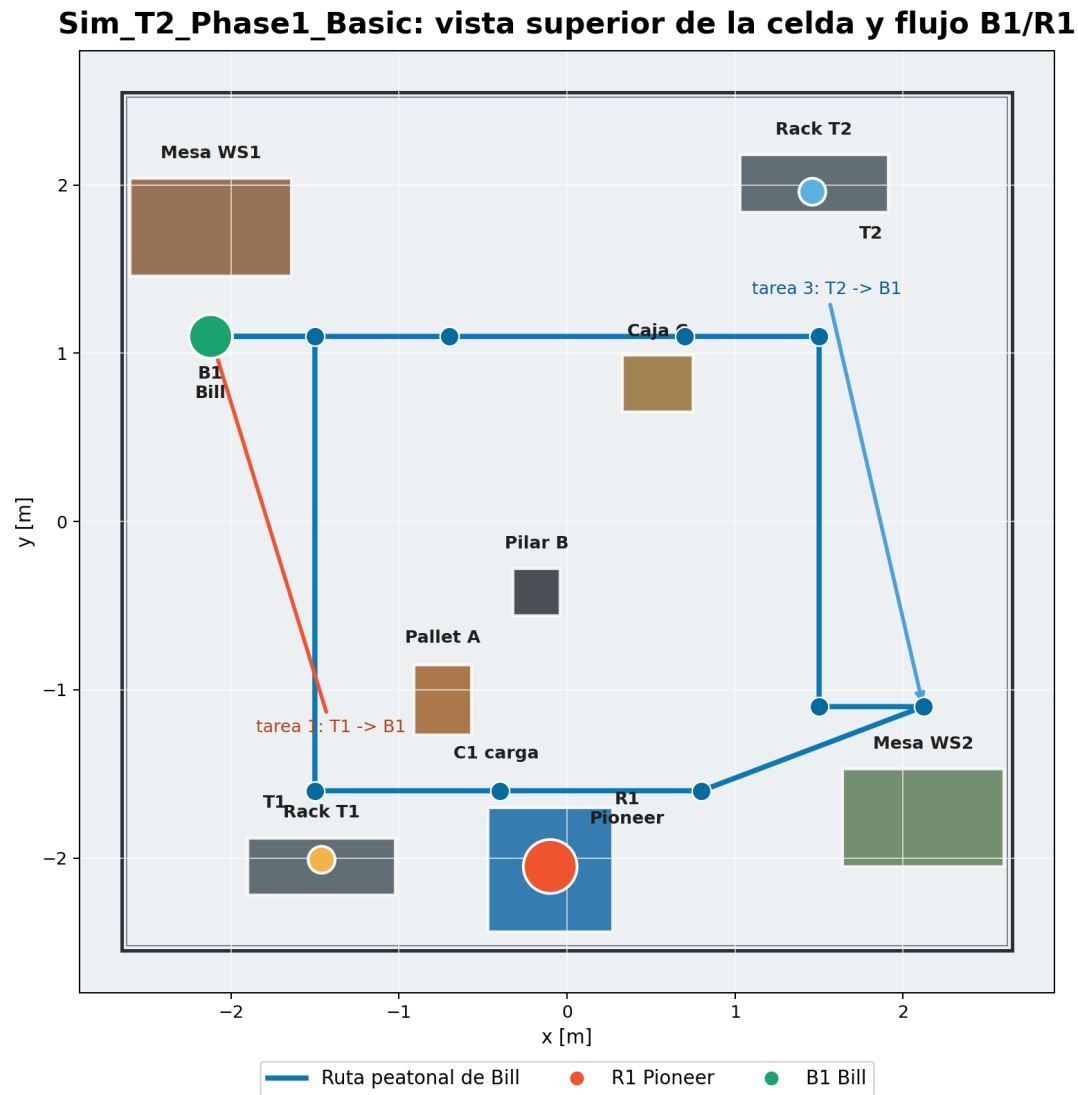


Figura 1.1: Vista superior de la celda base Sim_T2_Phase1_Basic.

2

Escenarios CoppeliaSim

2.1 Elementos del warehouse

La Tabla 2.1 lista los elementos principales de la celda de almacén.

Elemento	Alias en escena	Función
Robot móvil	/PioneerP3DX	R1 ejecuta la cola de tareas, navega, evita obstáculos, transporta herramientas y vuelve a carga.
Persona móvil	/B1, Bill	Modelo de persona de CoppeliaSim renombrado para actuar como People/mannequin; trabaja en dos mesas, camina entre WS1 y WS2 y solicita herramientas.
Herramienta/pieza 1	/T1	Pieza mecanizada visual, entregada en WS1 y devuelta a Rack_T1.
Herramienta/pieza 2	/T2	Pieza tipo placa electrónica/compuesta, entregada en WS2 y devuelta a Rack_T2.
Estación de carga	/C1	Punto final de retorno y recarga del robot.
Mesas de trabajo	/P1_WorkTable_1, /P1_WorkTable_2	Superficies donde Bill toma la pieza, la trabaja y la devuelve a R1.
Sofá de operador	/P1_Operator_Sofa_*	Elemento de descanso y referencia espacial dentro de la celda, ubicado fuera de la ruta principal.
Estanterías	/Rack_T1, /Rack_T2	Estaciones de almacenamiento opuestas para T1 y T2.
Obstáculos	Pallet, pilar y caja	Elementos detectables para activar evitación reactiva y mapeo.

Cuadro 2.1: Elementos de la celda de almacén simulada en CoppeliaSim.

2.2 Cómo funciona el Pioneer dentro de CoppeliaSim

El Pioneer P3DX de la escena se maneja como una plataforma diferencial con dos ruedas motrices laterales. Con las dos a la misma velocidad avanza recto, si una gira más rápido cambia de rumbo, y si

una avanza mientras la otra retrocede gira casi sobre su propio centro. El controlador calcula en cada ciclo una velocidad lineal v y una angular ω , las convierte en velocidades de rueda y las aplica a los motores de CoppeliaSim.

El ciclo de control en cada paso es:

1. Leer la pose estimada de R1, el estado de Bill, el objetivo de la tarea y los sensores de proximidad.
2. Actualizar el estimador y el mapa local con las detecciones del ciclo.
3. Decidir si el objetivo directo es seguro o si conviene pasar por un SLAM_WAYPOINT.
4. Calcular v y ω con atracción al objetivo, corrección de rumbo y repulsión de obstáculos.
5. Convertir v, ω a rueda izquierda y derecha, saturar los mandos y registrar telemetría.

Los cuatro módulos (percepción, estimación, planificación y control) producen una sola consigna de velocidad para el Pioneer.

2.3 Bill como actor móvil

Entre las dos estaciones, Bill pide herramientas y trabaja la pieza en la mesa correspondiente. Su gestor define las estaciones WS1/WS2, publica la estación y la herramienta actuales y anima las acciones de tomar, trabajar y entregar. La orientación se calcula con:

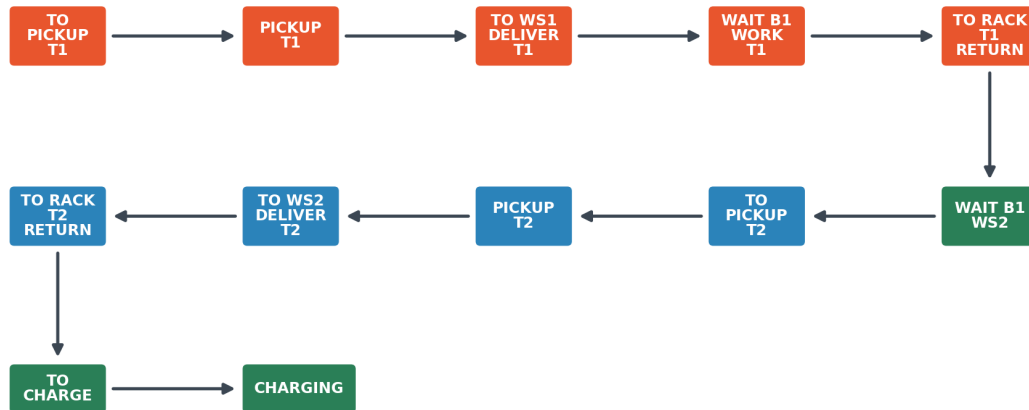
$$\theta_B = \text{atan2}(y_{k+1} - y_k, x_{k+1} - x_k)$$

Bill mira hacia donde camina. En las mesas se orienta hacia la superficie de trabajo, de acuerdo con la acción animada.

2.4 Cola de tareas

La cola de tareas se mueve con señales internas de CoppeliaSim y una máquina de estados dentro del controlador de R1. La Figura 2.1 resume la máquina de estados. R1 espera a que Bill termine T1 y llegue a WS2 antes de buscar T2; esta sincronización evita que la segunda entrega empiece antes de una solicitud válida.

Maquina de estados resumida del controlador R1



Cada flecha entra y sale de un estado: llegada por tolerancia, manipulación por temporizador y sincronización por señales de Bill.

Figura 2.1: Máquina de estados resumida del controlador R1 para entrega, trabajo de Bill, retorno de herramienta y carga.

2.5 Integración del código

El código de la escena cubre percepción, coordinación de tareas y registro de datos. El mannequin queda como persona móvil /B1; los sensores entregan datos a la evitación y al mapa. B1 tiene su propio gestor con tiempos de trabajo y movimiento, y los exportadores guardan CSV y JSON para revisar control, seguridad y SLAM con datos reproducibles.

Listing 2.1: Lectura de sensores y evitación reactiva en Lua. La variable `b1` es el handle de Coppeliasim del objeto /B1 (Bill); las detecciones sobre Bill se excluyen del mapa SLAM para no registrar al mannequin como obstáculo estático.

```

for _, s in ipairs(sensors) do
    local ok, distance, point, object = sim.readProximitySensor(s.handle)
    if ok and ok > 0 and distance > 0 then
        addSensorRay(s, distance, point, true)
        if object ~= b1 then -- b1 = handle de /B1 (Bill); se excluye del mapa SLAM
            registerSlamDetection(s, distance, point)
        end
        if distance < cfg.avoidRange then
            local influence = ((cfg.avoidRange - distance) / cfg.avoidRange)^2
            steer = steer - sign(s.y) * cfg.kRepulsion * influence
            slow = math.max(slow, influence)
        end
    else
        addSensorRay(s, cfg.sensorVizRange, nil, false)
    end
end
end
  
```

Listing 2.2: Fragmento de la máquina de estados de T1 y T2.

```

local TaskHandlers = {
    PICKUP_T1 = function()
        pickupTool(t1, 'T1', 'TO_WS1_DELIVER_T1')
    end,
    DELIVER_T1_WS1 = function()
        deliverToolToBill(t1, ws1WorkSurface, 1,
            'task1:{T1->B1@WS1}', 'WAIT_B1_WORK_T1')
    end,
    WAIT_B1_WS2_REQUEST_T2 = function()
        stopRobot('WAIT_B1')
        if readStringSignal('phase1B1Station', 'NONE') == 'WS2' then
            setTaskState('TO_PICKUP_T2')
        end
    end,
}

```

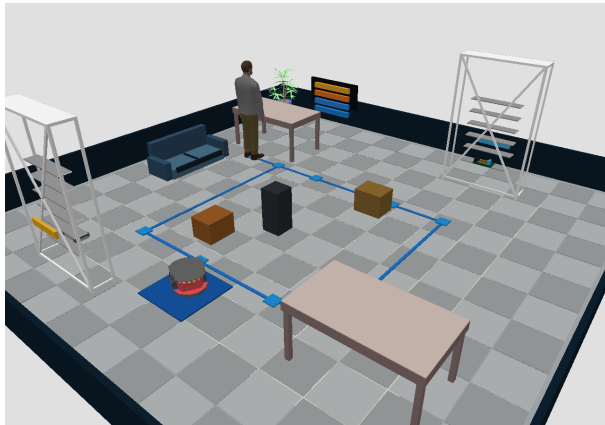
Listing 2.3: Selección de objetivo directo o punto intermedio SLAM.

```

local targetWorld = readWorldPosition(target)
local goalX, goalY = targetWorld[1], targetWorld[2]
local planX, planY = plannedWaypointTo(goalX, goalY, tolerance)
local dx, dy = planX - estX, planY - estY
local heading = normalizeAngle(atan2(dy, dx) - estTheta)
local obstacleSteer, obstacleSlow, minObstacle, hardStop = readObstacleField()
local v, w = computeCommand(distance, heading, lateral,
                            obstacleSteer, obstacleSlow, hardStop)
setWheelSpeeds(v, w)

```

2.6 Capturas de la simulación



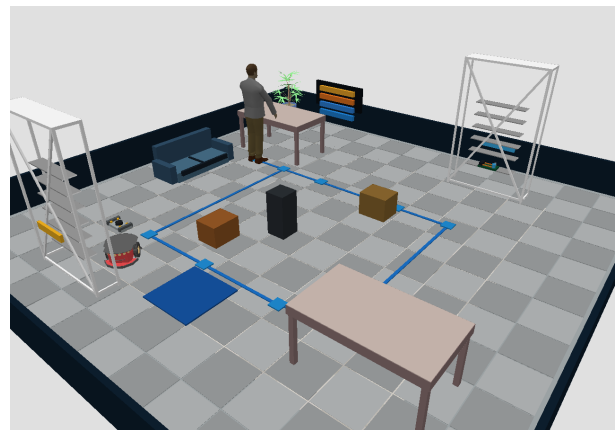
(a) Escena completa en CoppeliaSim.



(b) Rayos de sensor y evitación local.



(c) Bill trabajando frente a la mesa.



(d) Entrega de herramienta entre R1 y Bill.

Figura 2.2: Capturas 3D de la escena ejecutada en CoppeliaSim.

2.7 Piezas visuales

T1 y T2 se hicieron con primitivas para evitar geometrías genéricas. T1 lleva placa, nervios, boss central y pernos. T2 representa una placa compuesta/electrónica con raíles, conector y capacitor. La geometría diferenciada distingue cada herramienta durante la ejecución y en las capturas.

3

Modelo del robot y control

El Pioneer 3-DX se trata como robot diferencial [2]. El controlador calcula una velocidad lineal v y una velocidad angular ω , y de ahí salen las velocidades de rueda izquierda y derecha:

$$v_L = v - \frac{L}{2}\omega, \quad v_R = v + \frac{L}{2}\omega$$

Se adopta la convención usual de CoppeliaSim para esta escena: $\omega > 0$ incrementa el yaw del robot en sentido antihorario visto desde arriba. Con esa convención, la rueda derecha recibe mayor velocidad que la izquierda durante un giro positivo.

Si r es el radio de rueda, las consignas enviadas a los motores de CoppeliaSim son:

$$\omega_L = \frac{v_L}{r}, \quad \omega_R = \frac{v_R}{r}$$

En la implementación se usan $r = 0,0975$ m y $L = 0,331$ m, valores compatibles con el Pioneer 3-DX [3]. Las velocidades se saturan para mantener los comandos dentro de un rango físico y conservar la repetibilidad de las ejecuciones.

3.1 Seguimiento de objetivo

Para navegar hacia un rack, mesa o cargador, se calcula primero la posición del objetivo en coordenadas globales y luego el error respecto a la pose estimada del robot:

$$e_x = x_g - \hat{x}, \quad e_y = y_g - \hat{y}, \quad d = \sqrt{e_x^2 + e_y^2}$$

El rumbo deseado es:

$$\theta_g = \text{atan2}(e_y, e_x)$$

y el error angular usado por el controlador es:

$$e_\theta = \text{wrap}(\theta_g - \hat{\theta})$$

La consigna base se corrige con un término lateral y con una contribución de evitación de obstáculos:

$$v = f_d(d - d_f) \cos(e_\theta), \quad \omega = f_\theta(e_\theta) + f_{lat}(e_y^{body}) + f_{obs}$$

donde d_f es la distancia de seguimiento y f_{obs} resume el giro repulsivo derivado de sensores.

3.2 Modos de control evaluados

Se probaron cinco metodos: P, PI, PID, LQR y NMPC. Los controladores P, PI y PID se usan como referencia clásica junto con la estimación y la evitación. LQR y NMPC se incluyeron con alcance acotado. El LQR resuelve la ecuación de Riccati discreta *dentro de la propia escena*, en Lua (`solveDiscreteLqr`), de modo que la ganancia ya no está precalculada ni embebida; el solver in-scene se verifica fuera de línea contra `scipy.solve_discrete_are` (`compute_follower_lqr_gain.py`). En la misión de almacén se emplea una variante reactiva simplificada del LQR; el LQR de Riccati completo opera en el seguidor de Bill. El NMPC usa disparo directo y resuelve la secuencia de comandos con una búsqueda local de patrón (Hooke–Jeeves [4]), sin recurrir a IPOPT ni CasADi.

Modo	Descripción	Uso en la escena
P	Control proporcional sobre distancia y rumbo.	Estructura simple y estable, con ruta más larga (mayor duración en la ejecución medida).
PI	Agrega integral de distancia y rumbo.	El término integral añade sobrepaso; en la ejecución medida registra el mayor riesgo de obstáculo y el peor puntaje ponderado.
PID	Incluye derivadas para amortiguar saltos.	Mejor compromiso: obtiene el menor puntaje ponderado (menor riesgo, menor error de pose y menor duración) y se usa como modo base en las exportaciones SLAM.
LQR	Realimentación lineal de error longitudinal, lateral y rumbo.	Ecuación de Riccati discreta resuelta en la propia escena (Lua) para el seguidor de Bill; en el almacén se usa una variante reactiva simplificada.
NMPC	Control predictivo no lineal por disparo directo.	Evalúa secuencias de v, ω con restricciones y coste de suavidad mediante búsqueda local (véase Sección 4.5).

El puntaje ponderado que determina el modo ganador se calcula como la suma min-max normalizada de seis indicadores, con los pesos:

$$S = 0,28 s_{\text{mov}} + 0,18 s_{\text{bat}} + 0,20 s_{\text{var}} + 0,14 s_{\text{riesgo}} + 0,12 s_{\text{pose}} + 0,08 s_{\text{dur}}$$

donde menor puntaje es mejor. Cada indicador s_i viene en unidades distintas (segundos, julios, metros), así que antes de sumarlos se llevan al rango $[0, 1]$ con normalización min-max, $s_i = (x_i - x_{\min}) / (x_{\max} -$

$x_{\min}$), para que ninguno pese de más solo por su escala. Los pesos priorizan un seguimiento suave y eficiente: pesan más el tiempo en movimiento activo (0,28) y la suavidad de comandos (0,20) que el consumo (0,18), el riesgo (0,14), el error de pose (0,12) y la duración (0,08). Son ad hoc y calibrados para esta escena. Conviene tomar la ordenación como indicativa y no como un veredicto robusto: con $n = 1$ por modo y una normalización min-max sensible a los extremos, el ranking puede reordenarse si cambian las prioridades de la tarea o si se reajusta la navegación. De hecho, tras los últimos ajustes del controlador (tolerancias de aproximación, cotas del EKF y capas de recuperación) el modo de menor puntaje pasó a ser PID, con PI desplazado al final por registrar el mayor riesgo de obstáculo en su ejecución. Los valores coinciden con los definidos en `run_phase1_controller_comparison.py` (véase también el Anexo A.4). La Fig. 3.1 resume los resultados de los cinco modos.

Phase 1 Basic - Comparacion de controladores en la misma secuencia T1/T2

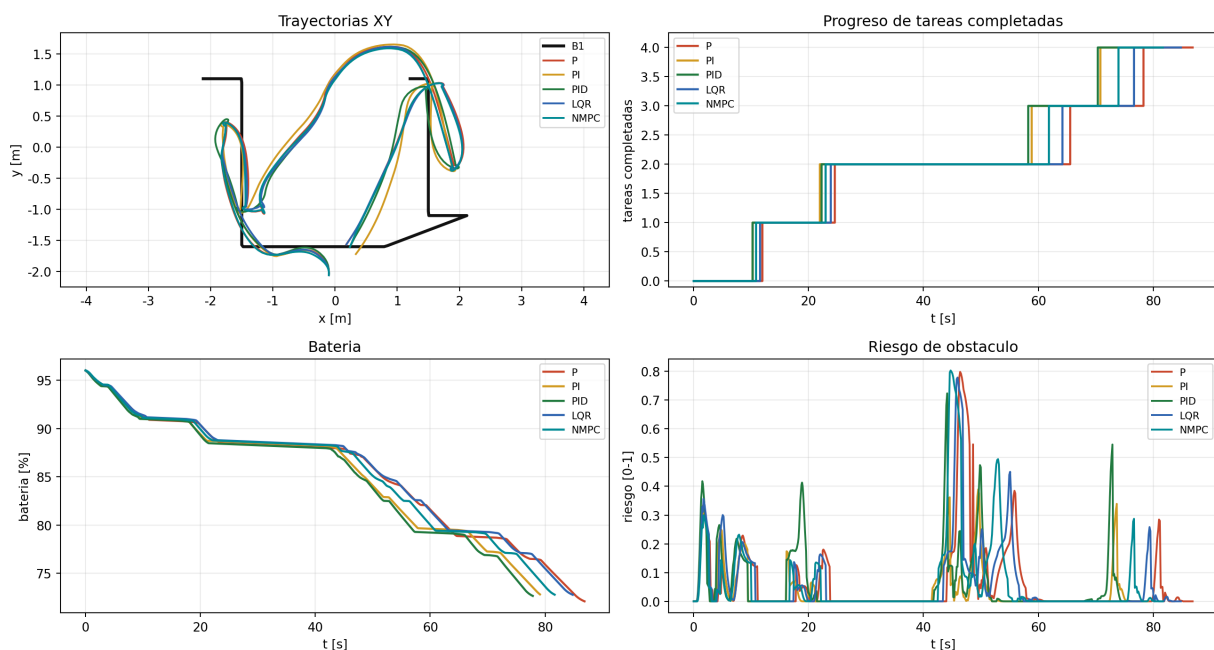


Figura 3.1: Comparación de modos de control exportada desde CoppeliaSim. Todos los modos completan la tarea. PID obtiene el menor puntaje ponderado con los pesos definidos ($n = 1$ por modo).

3.2.1 Capas de moldeado de velocidad sobre el modo base

El comando que produce la ley de control elegida (PID por defecto en el almacén) no se aplica en crudo: se le superponen varias capas que moldean la velocidad y que son *independientes del modo*, es decir, se aplican por igual a las cinco leyes, de modo que la comparación las mide en igualdad de condiciones. Se detallan en el capítulo de Fase 2 y se resumen aquí:

- **Factor de confianza (Fase 2).** En la celda desconocida la velocidad de avance se escala con la madurez del mapa SLAM (señal `phase1SpeedLearnFactor`, de $\approx 0,60$ a $\approx 0,88$): la primera pasada es cauta y las siguientes más ágiles. En Fase 1 el factor vale 1 y no interviene.
- **Aproximación a estanterías.** El robot reduce la velocidad en el tramo final hacia un rack para detenerse pegado sin embestirlo, y al salir retrocede un instante para liberarse antes de girar hacia el

siguiente objetivo.

- **Recuperación y geo-valla.** Si una maniobra se estanca, un arco de retroceso la desbloquea; y, como red de seguridad numérica, una geo-valla reincorpora el chasis a la celda si un artefacto del motor de física llegara a expulsarlo, además de las cotas por actualización que estabilizan el EKF.

Estas capas afectan *cómo* se ejecuta el comando del controlador en la navegación del almacén, pero no cambian *qué* ley de control gana la comparación, que sigue midiendo las cinco leyes en igualdad de condiciones.

4

Seguimiento de Bill y comparación de controladores

Antes de integrarlo en la celda completa, se probó el seguimiento de Bill en `Actividad2_1_Basic_Follower.ttt` (Fig. 4.1, capturas en Fig. 4.2). Bill camina con aceleración y frenado, y el Pioneer sigue un punto detrás de la persona. Si se apunta al centro del cuerpo, el robot se acerca demasiado. La referencia se sitúa detrás de Bill para mantener la separación deseada d_{des} .

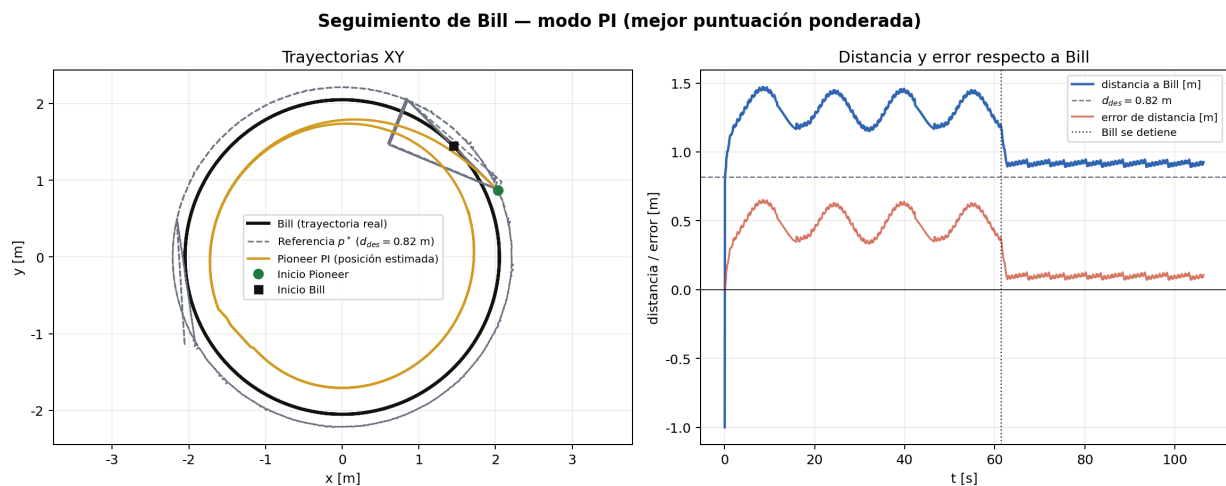


Figura 4.1: Escena de seguimiento: Bill define un objetivo móvil y R1 regula distancia, rumbo y suavidad de ruedas.

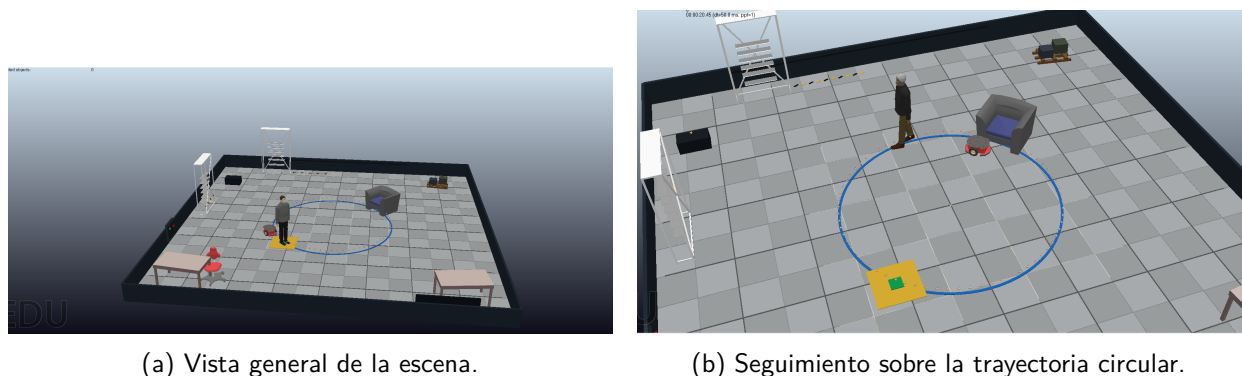


Figura 4.2: Capturas de ejecución de `Actividad2_1_Basic_Follower.ttt`: Bill recorre una trayectoria circular y R1 mantiene una distancia de seguimiento estable.

4.1 Geometría de seguimiento

Si $p_B = [x_B, y_B]^T$ es la posición de Bill, θ_B su orientación y d_{des} la separación deseada, el punto de referencia se calcula como:

$$p^* = p_B - d_{des} \begin{bmatrix} \cos \theta_B \\ \sin \theta_B \end{bmatrix}, \quad d_{des} = 0,82 \text{ m}.$$

El error de distancia se mide con respecto a Bill, mientras que el error de rumbo se calcula desde el robot hacia Bill (Fig. 4.3):

$$e_d = \|p_B - p_R\| - d_{des}, \quad e_\psi = \text{wrap}(\text{atan2}(y_B - y_R, x_B - x_R) - \theta_R).$$

Parte 1 follower: geometria de seguimiento Bill-Pioneer

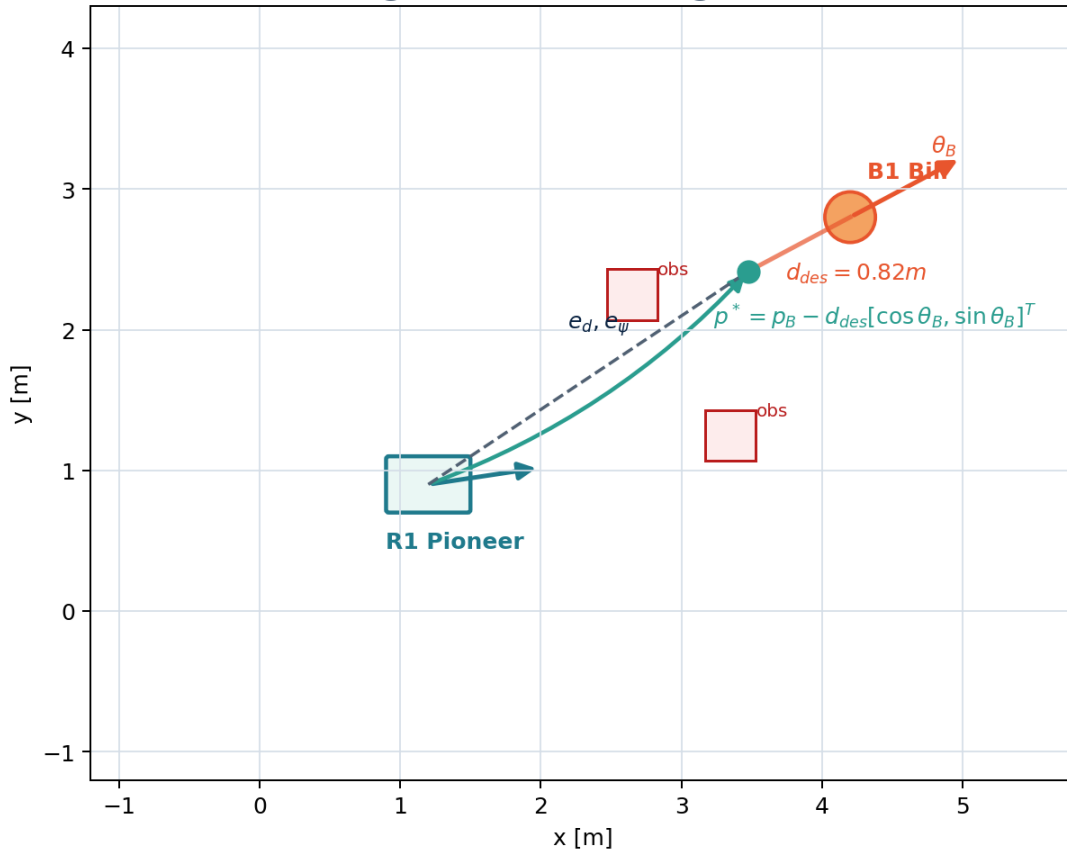


Figura 4.3: Referencia móvil del seguidor: el robot controla un punto detrás de Bill y no el centro de la persona.

4.2 Modelo cinemático usado por todos los modos

El movimiento plano del Pioneer se aproxima con el modelo unicycle:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega.$$

La conversión a velocidades angulares de rueda usa los parámetros del Pioneer 3DX definidos en el Cap. 3 ($r = 0,0975$ m, $L = 0,331$ m):

$$\omega_L = \frac{v - \frac{L}{2}\omega}{r}, \quad \omega_R = \frac{v + \frac{L}{2}\omega}{r}$$

Estas mismas señales se usan para calcular potencia aproximada, energía, variación de comando y suavidad de control.

La energía de la Tabla 4.1 se calcula como una métrica mecánica aproximada. Se obtiene en Lua con la potencia de avance de cada rueda, fricción de rodadura, un término viscoso de rueda y potencia de guiñada:

$$\begin{aligned} P_L &= \left| \frac{m}{2} a v_L \right| + \frac{F_r}{2} |v_L| + b_\omega \omega_L^2, \\ P_R &= \left| \frac{m}{2} a v_R \right| + \frac{F_r}{2} |v_R| + b_\omega \omega_R^2, \\ P_\theta &= |I_z \dot{\omega}|, \quad E = \sum_k (P_L + P_R + P_\theta) \Delta t. \end{aligned}$$

Los parámetros usados fueron $m = 16,5$ kg, $I_z = 0,72$ kg m², $F_r = 2,8$ N y $b_\omega = 0,006$ kg m² s⁻¹ (amortiguamiento viscoso rotacional, unidades SI: N m s). En esa expresión, $a = \dot{v}$ es la aceleración lineal estimada entre muestras y $\dot{\omega}$ es la aceleración angular. Esta magnitud sirve como indicador relativo entre controladores en la misma escena (estimación cinemática aproximada, no eléctrica).

Los términos $\frac{m}{2} a v_L$ y $\frac{m}{2} a v_R$ reparten aproximadamente la potencia inercial lineal, no como un modelo dinámico completo de rueda. La inercia real actúa sobre el centro de masa y se transmite por los contactos; se reparte por rueda solo para obtener una métrica relativa sensible a frenazos, inversiones de sentido y oscilaciones de comando.

4.3 P, PI y PID

Los controladores clásicos se aplican sobre e_d y se combinan con una realimentación de rumbo:

$$\begin{aligned} u_P &= K_p e_d, \quad u_{PI} = K_p e_d + K_i \sum_{k=0}^n e_d(k) \Delta t, \\ u_{PID} &= K_p e_d + K_i \sum_{k=0}^n e_d(k) \Delta t + K_d \frac{e_d(k) - e_d(k-1)}{\Delta t}. \end{aligned}$$

El término proporcional actúa sobre el error instantáneo y el derivativo amortigua sus cambios bruscos; el integral elimina el error de régimen permanente, esté el objetivo en movimiento o detenido. En ejecución, todos los modos añaden una corrección de evitación de obstáculos al ángulo de giro:

$$\omega = \omega_{\text{controlador}} + \omega_{\text{obs}},$$

donde ω_{obs} es el término repulsivo de la sección de campos potenciales (§5). En todos los modos se

aplican saturaciones de velocidad y aceleración.

4.4 LQR discreto

El modo LQR del seguidor regula el error local alrededor del punto p^* , siguiendo la forma clásica de control cuadrático lineal [5]. El estado se define en el marco del robot mediante la transformación:

$$\begin{aligned} e_{long} &= \cos \theta_R (x_{p^*} - x_R) + \sin \theta_R (y_{p^*} - y_R), \\ e_{lat} &= -\sin \theta_R (x_{p^*} - x_R) + \cos \theta_R (y_{p^*} - y_R), \quad e_{yaw} = \text{wrap}(\psi_{ref} - \theta_R), \end{aligned}$$

donde (x_{p^*}, y_{p^*}) es el punto de referencia p^* . El vector de estado y la acción de control son:

$$x_L = \begin{bmatrix} e_{long} & e_{lat} & e_{yaw} \end{bmatrix}^T, \quad u = \begin{bmatrix} \Delta v & \Delta \omega \end{bmatrix}^T.$$

Con $v_{ref} = 0,22$ m/s, radio local de giro $\rho_{ref} = 1,55$ m, $\omega_{ref} = v_{ref}/\rho_{ref}$ y $\Delta t = 0,05$ s, A y B salen de derivar la dinámica del error en el marco de Frenet (cómo cambian $e_{long}, e_{lat}, e_{yaw}$ al desviarse de la trayectoria de referencia); los términos $\pm \omega_{ref}$ y v_{ref} son el acoplamiento entre esos tres errores al recorrer una curva:

$$A = \begin{bmatrix} 0 & \omega_{ref} & 0 \\ -\omega_{ref} & 0 & v_{ref} \\ 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} -1 & 0 \\ 0 & 0 \\ 0 & -1 \end{bmatrix}.$$

La segunda fila de B es $[0, 0]$ porque en la linealización del unicycle alrededor de la trayectoria de referencia en el marco de Frenet, el error lateral satisface $\dot{e}_{lat} = v_{ref}e_{yaw} - \omega_{ref}e_{long}$: ninguna entrada $(\Delta v, \Delta \omega)$ actúa directamente sobre e_{lat} . El error lateral solo se puede reducir indirectamente, a través del canal de error de rumbo e_{yaw} , lo que el LQR compensa asignando un peso alto a e_{lat} en la matriz Q ($Q_{22} = 80$).

$$A_d = I + \Delta t A, \quad B_d = \Delta t B,$$

discretización de Euler hacia adelante con paso Δt . La ganancia K se calcula *dentro de la escena* al arrancar: partiendo de $P = Q$ se aplica repetidamente la ecuación de Riccati en diferencias $P \leftarrow A_d^\top P A_d - (A_d^\top P B_d)(R + B_d^\top P B_d)^{-1}(B_d^\top P A_d) + Q$. Como el par (A_d, B_d) es controlable, P converge a un valor estable; en ese punto fijo se cumple la ecuación algebraica de Riccati y la ganancia óptima estacionaria es $K = (B_d^\top P B_d + R)^{-1} B_d^\top P A_d$, sin valores precargados. Las matrices de coste son:

$$Q = \text{diag}(35, 80, 18), \quad R = \text{diag}(6, 3),$$

$$K = (B_d^T P B_d + R)^{-1} B_d^T P A_d.$$

Los pesos proceden de una calibración empírica sobre el seguimiento de Bill, no de una identificación formal del Pioneer. Se penalizó más el error lateral ($q_{lat} = 80$) que el longitudinal ($q_{long} = 35$): un adelanto o retraso longitudinal es tolerable, pero un cruce lateral con la referencia móvil no. El coste de giro en R se dejó menor que el de avance para que el regulador corrija antes el rumbo que la velocidad.

La ley aplicada en Lua usa $u = -Kx_L$, con saturación posterior de v y ω .

La matriz resultante calculada en escena fue:

$$K = \begin{bmatrix} -2.3496 & 1.2925 & 0.1203 \\ 0.2427 & -4.3570 & -2.6887 \end{bmatrix}.$$

4.5 NMPC por disparo directo

El modo NMPC se implementa como control predictivo no lineal por disparo directo. Mantiene una secuencia de v_k, ω_k , simula el modelo no lineal hacia adelante sobre el horizonte y optimiza esa secuencia con una búsqueda de patrón de Hooke-Jeeves [4], siguiendo la estructura general de MPC [6].

$$x_{k+1} = x_k + v_k \cos(\theta_k) \Delta t, \quad y_{k+1} = y_k + v_k \sin(\theta_k) \Delta t,$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t.$$

La función de coste usada por el script penaliza error de distancia, error al punto p^* , rumbo, esfuerzo, potencia de rueda y variación de comando:

$$\begin{aligned} J = \sum_{k=0}^{N-1} & (q_d e_d(k))^2 + q_t \|p_k^* - p_k\|^2 + q_\psi e_\psi(k)^2 \\ & + r_v v_k^2 + r_\omega \omega_k^2 + r_{whl} (\omega_{L,k}^2 + \omega_{R,k}^2) \\ & + s_v \Delta v_k^2 + s_\omega \Delta \omega_k^2 + q_f \|p_N^* - p_N\|^2. \end{aligned}$$

El coste incluye dos términos de posición con roles distintos: $q_d e_d^2$ penaliza el error escalar de distancia a Bill (mantiene el radio correcto respecto al objetivo, pero no corrige la desviación lateral); $q_t \|p^* - p\|^2$ penaliza la distancia euclídea al punto p^* , introduciendo una corrección lateral suave que alinea el robot detrás de Bill. Si se quita q_t , el robot conserva el radio pero se desvía de lado; quitar q_d deja el radio en manos solo de p^* , con menos robustez ante errores de predicción de Bill. El valor $q_t = 7,0 \ll q_d = 48,0$ asegura que la corrección lateral no interfiera con la regulación de distancia.

Los pesos fueron $q_d = 48,0$, $q_t = 7,0$, $q_\psi = 1,2$, $r_v = 1,4$, $r_\omega = 1,8$, $r_{whl} = 0,020$, $s_v = 65,0$, $s_\omega = 18,0$ y $q_f = 6,0$, con horizonte $N = 10$, $\Delta t = 0,22$ s y hasta seis pasadas de refinamiento por ciclo con pasos geoméricamente decrecientes. Este controlador opera en la escena de seguimiento de Bill. La escena

warehouse usa un controlador de tarea distinto, cuyo modo base es PID y que también ofrece los cinco modos, pero en *variantes reactivas simplificadas* (ganancias fijas para LQR y un coste predictivo de un solo paso para NMPC), adaptadas a un objetivo de navegación estático en lugar del punto móvil p^* detrás de Bill. El regulador LQR con ganancia de Riccati calculada en escena y el NMPC de horizonte completo con búsqueda de patrón se demuestran en el *follower*, donde el objetivo móvil ejercita mejor la respuesta dinámica; por eso la comparación cuantitativa de controladores de este capítulo usa el follower.

La búsqueda de patrón es determinista: la secuencia se inicializa con los comandos del ciclo anterior desplazados un paso (*warm start*), y en cada pasada se prueban perturbaciones $\pm\Delta v$ y $\pm\Delta\omega$ sobre cada elemento del horizonte, aceptando solo las que reducen J . Los pasos parten de $\Delta v = 0,090$ y $\Delta\omega = 0,220$ y se reducen por un factor 0,55 cuando una pasada completa no mejora el coste, hasta alcanzar un suelo de 0,004 y 0,010. El coste es predictivo y el modelo es no lineal; el solucionador es una búsqueda de patrón de Hooke-Jeeves, no un método SQP ni de punto interior.

Los límites de actuación se aplican durante la búsqueda:

$$v_k \in [-v_{rev,max}, v_{max}], \quad \omega_k \in [-\omega_{max}, \omega_{max}].$$

Esta versión conserva el modelo no lineal y el coste predictivo, pero limita la optimización a una búsqueda local de vecindario sobre comandos.

4.6 Resultados de seguimiento

Modo	Error mov. [m]	Error final [m]	Energía [J]	RMS Δu	Flips
P	0,3537	0,1017	49,495	0,0490	18
PI	0,2010	0,0782	64,146	0,1356	18
PID	0,1912	0,0457	83,181	0,1310	16
LQR	0,0167	0,0762	87,314	0,0791	4
NMPC	0,0189	0,0484	73,415	0,0457	3

$n=1$ por modo. Energía: estimación cinemática aproximada (véase §4.2). Indicadores cualitativos.

Cuadro 4.1: Métricas del seguidor de Bill por modo de control.

Actividad 2.1 - Pioneer siguiendo a Bill en círculo: P, PI, PID, LQR y NMPC

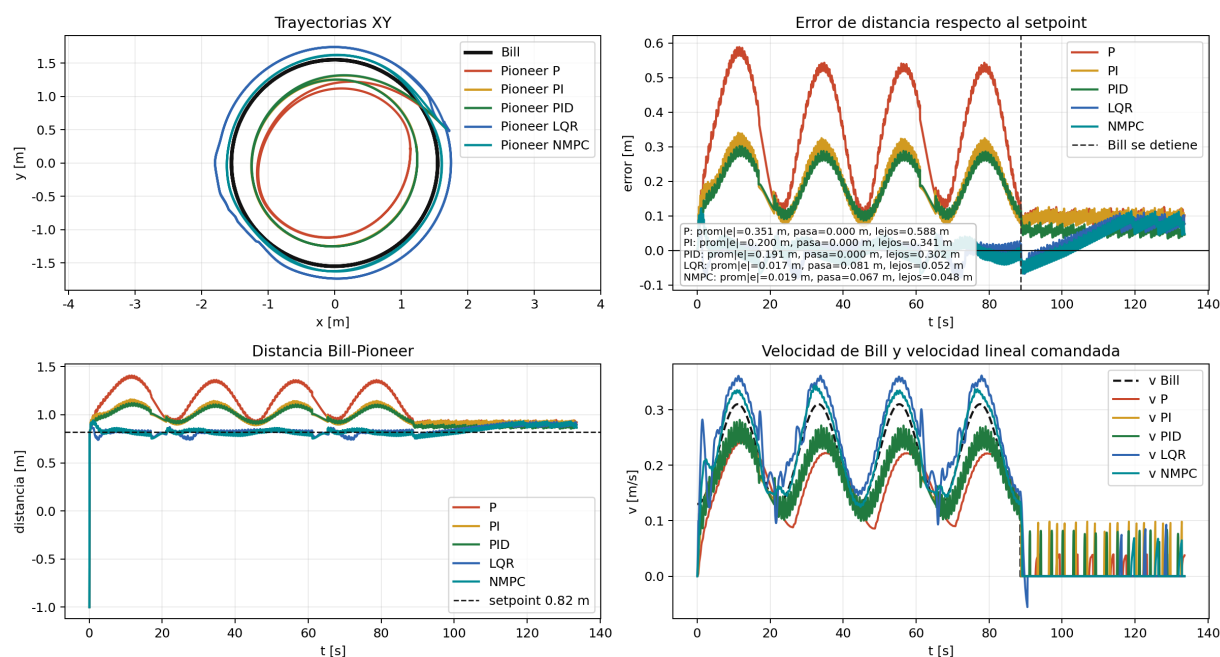


Figura 4.4: Comparación temporal de P, PI, PID, LQR y NMPC en la escena de seguimiento de Bill.

Actividad 2.1 - Datos de ruedas y potencia estimada

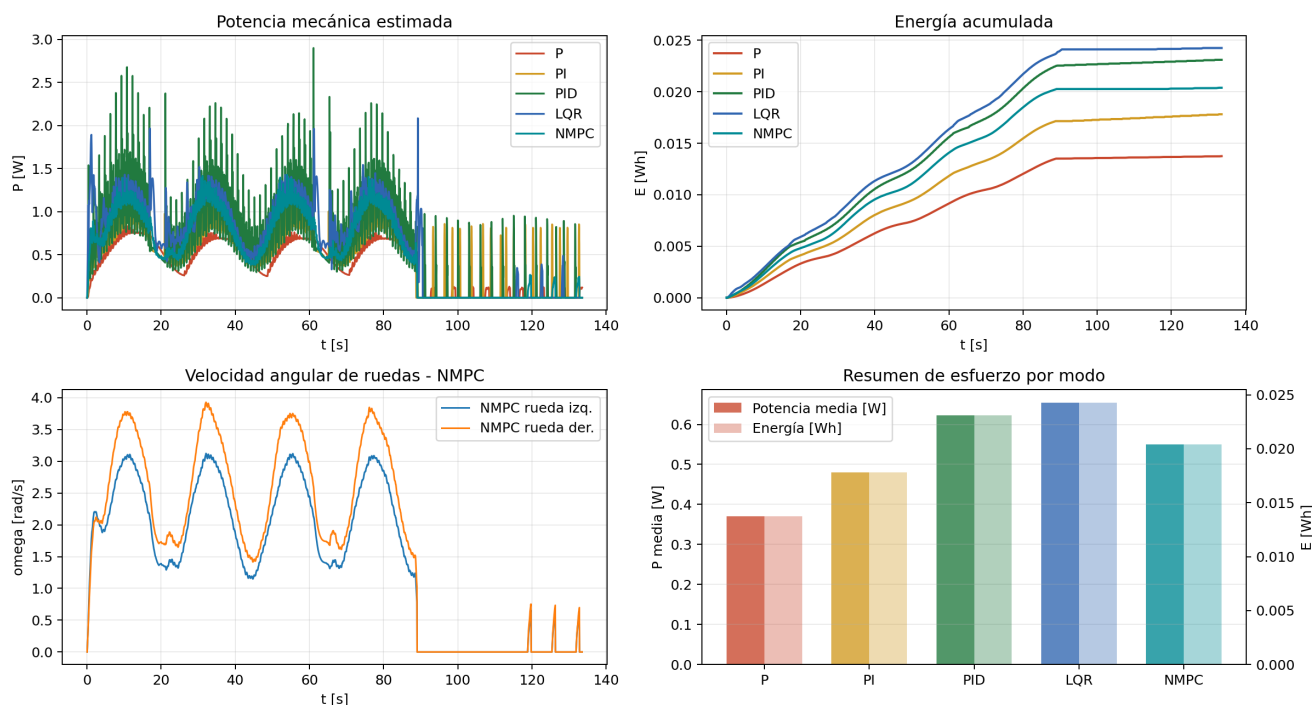


Figura 4.5: Coste físico aproximado del seguidor: potencia instantánea y energía acumulada en las ruedas.

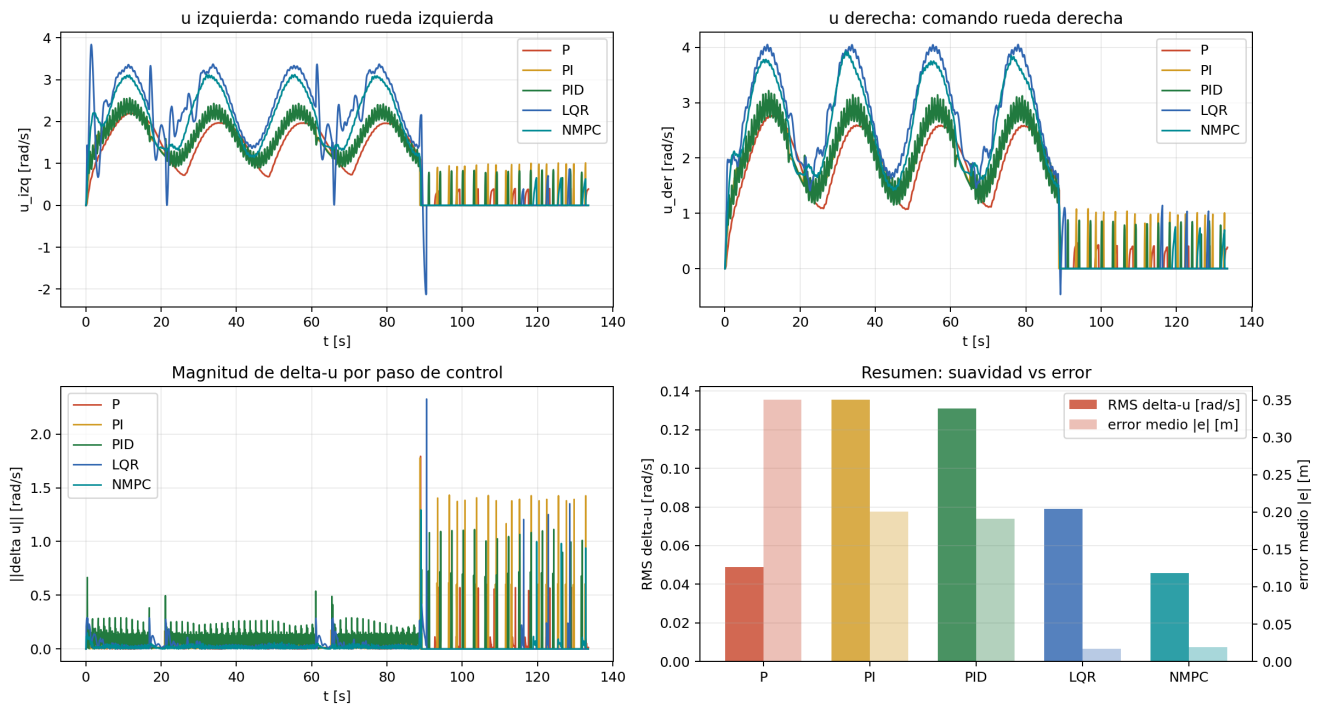
Actividad 2.1 - Suavidad de comandos de rueda u y Δu 

Figura 4.6: Suavidad de comandos del seguidor: variación de las órdenes de rueda y cambios de sentido.

La Fig. 4.4 muestra la respuesta temporal de los cinco modos; la Fig. 4.5 compara potencia y energía de rueda; la Fig. 4.6 compara la suavidad de comandos. P gasta menos energía y deja más error con Bill en movimiento. PID ajusta bien el error final a costa de más variación de comando y más energía. LQR y NMPC reducen el error medio durante el seguimiento; NMPC es el más suave en inversiones de sentido y RMS de Δu , porque su coste penaliza la variación de comando y el esfuerzo de rueda.

5

Campos potenciales, evitación de obstáculos y planificación local

Hacia Bill (usado como mannequin) actúa un campo atractivo; sobre él se superponen la repulsión por obstáculos y una capa local de mapa [7]. Los sensores de proximidad reducen la velocidad y corrigen el giro cuando se detecta un obstáculo al frente o en los laterales. El mapa SLAM entra cuando la ruta directa se bloquea y se usa un punto intermedio antes de volver al objetivo.

La palabra “campo” no significa que CoppeliaSim aplique una fuerza física al Pioneer. Es una forma de construir un vector de decisión: el objetivo tira del robot hacia Bill y cada obstáculo cercano empuja el rumbo hacia el lado libre. El resultado se traduce después a v y ω , que sí son las señales que acaban moviendo las ruedas.

5.1 Campo atractivo hacia Bill

El campo atractivo apunta al mannequin, que en esta escena es Bill. Si $p_R = [x_R, y_R]^T$ es la posición estimada del robot y $p_B = [x_B, y_B]^T$ la posición de Bill, se usa el potencial atractivo:

$$U_{att}(p_R) = \frac{1}{2}k_{att}\|p_B - p_R\|^2, \quad F_{att} = -\nabla U_{att} = k_{att}(p_B - p_R).$$

La fuerza atractiva se traduce a consignas de avance y giro:

$$\theta_{att} = \text{atan2}(y_B - y_R, x_B - x_R), \quad e_\theta = \text{wrap}(\theta_{att} - \theta_R),$$

$$v_{att} = \text{sat}_{[0, v_{max}]}(k_v(\|p_B - p_R\| - d_f)), \quad \omega_{att} = k_\theta e_\theta.$$

La repulsión de obstáculos modifica ω_{att} y reduce v_{att} ; el objetivo de seguir a Bill no cambia.

5.2 Campo reactivo de obstáculos

Para cada sensor con detección positiva se calcula una influencia normalizada. La función de influencia cuadrática sigue el modelo de decaimiento de los campos potenciales repulsivos de Khatib [8]: la fuerza crece al cuadrado al acercarse al obstáculo y se anula en el rango de influencia. La asignación de esa fuerza a velocidades angulares por posición lateral del sensor es el esquema Braitenberg vehicle-2b [9].

$$\alpha_i = \begin{cases} \left(\frac{d_0 - d_i}{d_0}\right)^2, & 0 < d_i < d_0 \\ 0, & d_i \geq d_0 \end{cases}$$

donde d_i es la distancia medida y d_0 el rango de influencia. La contribución angular se calcula ponderando por la posición lateral del sensor:

$$\omega_{obs} = - \sum_i \text{sgn}(y_i) k_{rep} \alpha_i w_i$$

El factor w_i es el peso de frontalidad del sensor i , definido por su coordenada x en el marco del robot:

$$w_i = \begin{cases} 1,00 & \text{si } s_{x,i} > 0,02 \text{ m} & (\text{sensores frontales}) \\ 0,26 & \text{si } |s_{x,i}| \leq 0,04 \text{ m} & (\text{sensores laterales}) \\ 0,18 & \text{si } s_{x,i} < -0,04 \text{ m} & (\text{sensores traseros}) \end{cases}$$

El signo de y_i indica la dirección lateral del sensor en el marco del robot. Si un obstáculo está dentro de una distancia crítica frontal, el robot entra en modo AVOIDING, reduce v y gira en el sentido de menor riesgo.

La convención de signo usada en el script es la habitual del modelo unicycle: $\omega > 0$ incrementa el yaw en sentido antihorario y aumenta la velocidad de la rueda derecha respecto de la izquierda. Con $y_i > 0$ para sensores del lado izquierdo, el signo negativo de ω_{obs} hace que un obstáculo a la izquierda produzca giro horario, es decir, alejamiento hacia la derecha. Para sensores del lado derecho ocurre lo contrario.

5.3 Visualización de sensores

Los 16 sensores ultrasónicos del Pioneer se dibujan como rayos. Sin impacto, el rayo llega al rango nominal; con obstáculo, termina en el punto detectado. Esta visualización relaciona detección, giro reactivo y separación frente a pallet, pilar o caja.

5.4 Punto intermedio de planificación SLAM

En esta fase se usa una regla local de punto intermedio. Si un landmark o una celda ocupada cae demasiado cerca del segmento directo entre la pose estimada y el objetivo, se generan candidatos laterales a distancia d_{off} . Se elige el candidato con menor coste:

$$J(q) = \|q - \hat{x}_{R1}\| + \|x_g - q\| + \lambda C_{obs}(q)$$

donde C_{obs} penaliza candidatos cercanos a obstáculos mapeados. Si se activa este desvío, la señal `phase1PlannerMode` toma el valor `SLAM_WAYPOINT`; al alcanzar ese punto, el sistema vuelve a ruta directa.

6

Estimación y SLAM

El estimador alterna predicción y corrección. Primero propaga la pose con odometría diferencial; después la corrige con la información de los sensores (actualización EKF de rango y rumbo en KALMAN_LANDMARK, o ajuste *scan-to-map* en las variantes de grid) y lleva las lecturas de sensor a coordenadas del warehouse. Una referencia global débil y ruidosa (tipo baliza UWB) actúa además como ancla de baja tasa para mantener observable el marco global. Con esas detecciones se actualiza un mapa compacto de landmarks o una grid de ocupación, según el algoritmo seleccionado [10].

La pose que entrega CoppeliaSim tiene seis componentes: $[x, y, z, \alpha, \beta, \gamma]$. El estimador usa solo la proyección plana $[x, y, \theta]$, con $\theta = \gamma$. La altura, roll y pitch se descartan porque el Pioneer permanece sobre el piso de la celda y la tarea se formula en 2D.

6.1 Sensores: LiDAR, ultrasonidos y rayos de proximidad

Un LiDAR 2D real emite muchos pulsos láser en ángulos conocidos. Para cada ángulo recibe una distancia, normalmente por tiempo de vuelo, y forma un escaneo polar:

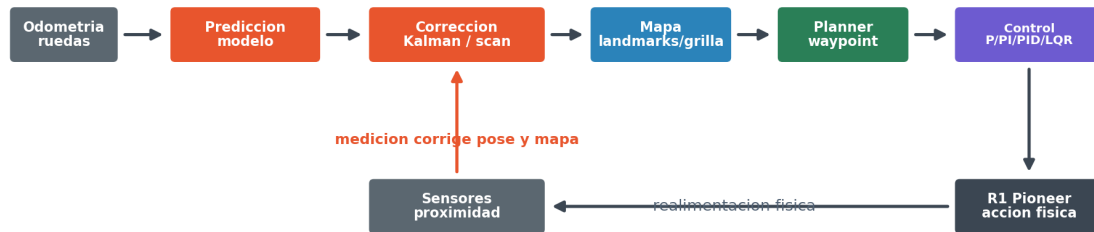
$$\mathcal{S}_k = \{(r_i, \beta_i)\}_{i=1}^N.$$

Cada par (r_i, β_i) da la distancia al primer obstáculo en ese ángulo. Con la pose del robot, ese punto se transforma al marco global:

$$p_i^W = \begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix} + R(\hat{\theta}) \begin{bmatrix} r_i \cos \beta_i \\ r_i \sin \beta_i \end{bmatrix}.$$

El Pioneer usado aquí no lleva un LiDAR físico 2D. La escena usa los 16 sensores ultrasónicos/proximidad del modelo P3DX, dibujados como rayos para que se vea qué detecta cada sensor. Conceptualmente se tratan igual que un escaneo de baja resolución: cada sensor tiene un ángulo fijo β_i , devuelve una distancia r_i si hay impacto y se transforma al mapa con la pose estimada. La diferencia está en la densidad: un LiDAR aporta muchos más rayos y contornos más continuos. El arreglo del Pioneer tiene pocas direcciones, de modo que los mapas son aproximaciones locales. Las variantes GMapping, Hector y Cartographer se implementan sobre esos rayos de proximidad simulados para comparar representaciones de mapa, sin equivalencia con sus paquetes ROS completos. La Fig. 6.1 muestra el ciclo completo desde sensores hasta control.

Ciclo estimacion - mapa - planificacion - control



Flujo principal: odometría -> estimación -> mapa -> planner -> control -> acción. La vuelta cerrada regresa por sensores.

Figura 6.1: Secuencia de estimación, mapa, planificación local y control de R1.

6.2 Datos usados para comparar SLAM

Los cuatro métodos reciben las mismas entradas: odometría diferencial, pose ruidosa de referencia para cerrar el estimador, rayos de proximidad y el mismo objetivo de navegación. La comparación evalúa cuatro representaciones implementadas en Lua para la misma celda: rasgos compactos, grid de ocupación, ajuste local por scan matching y submapas. El alcance difiere del de los paquetes ROS completos.

Las cuatro variantes reciben las mismas entradas y el mismo ruido: idéntica odometría diferencial, la misma referencia global débil con ruido gaussiano ($\sigma = 0,030$ m en x, y y $0,018$ rad en yaw, con ganancias $k_{xy} = 0,14$ y $k_{\theta} = 0,20$) y los mismos rayos de proximidad. Lo que cambia es cómo usa cada método la información del sensor para corregir la pose y construir el mapa:

- **KALMAN_LANDMARK**: actualización EKF de la pose por rango y rumbo a cada landmark asociado, con gating de Mahalanobis (umbral $\chi^2_2 = 9,0$) para descartar asociaciones erróneas y peso de corrección 0,35.
- **GMAPPING**: solo grid de ocupación; la pose proviene de odometría más el ancla global.
- **HECTOR** y **CARTOGRAPHER**: corrección de pose por ajuste *scan-to-map* Gauss-Newton sobre la grid log-odds (ganancias de mezcla 0,65 y 0,55).

La comparación es por tanto equitativa en ruido: las diferencias de RMSE¹ reflejan la representación de mapa y el método de corrección, no un ruido inyectado distinto por método.

¹El RMSE se mide contra la referencia global ruidosa (pose del simulador más ruido gaussiano), no contra una verdad de terreno; es una medida de consistencia del estimador bajo condiciones reproducibles, no de localización absoluta.

La covarianza de medición de la actualización EKF se parametriza con $\sigma_r = 0,075\text{ m}$ y $\sigma_\beta = 0,10\text{ rad}$ para rango y rumbo. Esto mantiene repetible la comparación; no pretende modelar el ruido exacto de un sensor ultrasónico real.

Cada algoritmo guarda algo distinto y eso cambia lo que recibe el planificador: Kalman-landmark, pocos puntos fusionados; GMapping, probabilidades por celda; Hector, la corrección de pose por ajuste scan-grid; Cartographer, submapas con cierres de ciclo. Como un landmark y una celda ocupada no significan lo mismo, la comparación no se reduce a contar rasgos.

6.3 Por qué hace falta Kalman

La odometría diferencial predice bien durante pocos instantes, pero acumula error: un pequeño desajuste de rueda, saturación o giro se convierte en deriva de x, y, θ . Las mediciones de sensor tampoco son perfectas; un impacto puede aparecer desplazado por ruido, discretización o por estar mirando un borde. El filtro de Kalman se usa para combinar esas dos fuentes en vez de creer ciegamente en una sola.

Primero se predice la pose con el modelo de movimiento y los comandos de rueda; luego, ante una medición ruidosa, se calcula la corrección. La ganancia K pesa la medición según la incertidumbre: con P alta corrige más, y con R poco fiable corrige menos. En la escena esto mantiene una pose suave para el controlador y evita que un rayo aislado mueva el mapa de forma brusca.

En esta actividad Kalman es la opción base porque encaja con una celda pequeña y pocos obstáculos: mantiene una pose estable, fusiona detecciones cercanas como landmarks y entrega al planificador una lista compacta de puntos peligrosos. Su límite: no reconstruye paredes ni superficies completas como una grid de ocupación.

La corrección combina dos fuentes y la del sensor domina. La principal es la observación de los sensores de proximidad: cada vez que una lectura se asocia a un landmark del mapa, una actualización EKF corrige la pose por rango y rumbo. El jacobiano de observación H es la derivada de la función que, dada la pose, predice el rango y el rumbo al landmark; por eso es 2×3 (dos medidas frente a las tres componentes de la pose). La covarianza de innovación $S = HPH^\top + R$ es la incertidumbre esperada de esa predicción (la de la pose llevada al sensor, más el ruido R); dividir por S en $K = PH^\top S^{-1}$ hace que una innovación poco fiable pese menos. Antes de aplicar la corrección, un *gate* de Mahalanobis filtra las asociaciones malas: esa distancia mide cuánto se aparta la innovación pesándola con S^{-1} y, al tener dos componentes (rango y rumbo), sigue una χ^2 con 2 grados de libertad; el umbral 9,0 equivale a un nivel de confianza cercano al 98 %, así que por encima la asociación se descarta. Esto evita el fallo típico que hace diverger un EKF-SLAM con sónares dispersos. El peso de corrección aplicado es 0,35. La segunda fuente es una referencia global débil con ruido gaussiano genuino, aplicada con ganancia pequeña solo para mantener observable el marco global (en especial el yaw) en misiones largas. Trabajar con la covarianza 3×3 de la pose, en vez de una covarianza conjunta pose-mapa, es lo que evita la divergencia con un anillo de 16 ultrasonidos; coincide con el alcance de KALMAN_LANDMARK, que no mantiene la covarianza conjunta de un EKF-SLAM completo [10].

6.4 Predicción de pose

Con velocidades odométricas v_k, ω_k , el modelo diferencial discreto es:

$$\begin{aligned}\hat{x}_k^- &= \hat{x}_{k-1}^+ + v_k \cos(\hat{\theta}_{k-1}^+) \Delta t \\ \hat{y}_k^- &= \hat{y}_{k-1}^+ + v_k \sin(\hat{\theta}_{k-1}^+) \Delta t \\ \hat{\theta}_k^- &= \text{wrap}(\hat{\theta}_{k-1}^+ + \omega_k \Delta t)\end{aligned}$$

La incertidumbre también se propaga. Para ello se deriva el modelo de movimiento respecto a la pose anterior: ese jacobiano G_k tiene como únicos términos no triviales $-v_k \Delta t \sin \theta$ y $v_k \Delta t \cos \theta$, que miden cuánto se desplazan x e y ante un pequeño error de orientación. La covarianza predicha es entonces $P_k^- = G_k P_{k-1}^+ G_k^\top + Q_k$ (la del paso anterior arrastrada por G_k , más el ruido de proceso Q_k):

$$G_k = \begin{bmatrix} 1 & 0 & -v_k \Delta t \sin \hat{\theta}_{k-1}^+ \\ 0 & 1 & v_k \Delta t \cos \hat{\theta}_{k-1}^+ \\ 0 & 0 & 1 \end{bmatrix}, \quad P_k^- = G_k P_{k-1}^+ G_k^\top + Q_k$$

6.5 Corrección Kalman-landmark

La formulación de referencia sigue el filtro de Kalman extendido (EKF) para sistemas de navegación con landmarks [10]:

$$K_k = P_k^- H_k^\top (H_k P_k^- H_k^\top + R_k)^{-1}$$

$$\hat{x}_k^+ = \hat{x}_k^- + K_k (z_k - H_k \hat{x}_k^-), \quad P_k^+ = (I - K_k H_k) P_k^-$$

En la escena, esta actualización corrige la pose del robot a partir de cada landmark observado (rango y rumbo), con el *gate* de Mahalanobis que rechaza asociaciones improbables. Las detecciones se asocian por cercanía: si una lectura cae dentro del radio de asociación de un landmark, su posición se refina y la pose se corrige; las lecturas no asociadas crean landmarks nuevos, hasta el máximo configurado. La posición de un landmark nuevo se obtiene invirtiendo la observación: a partir de la pose y del par (rango, rumbo) se calcula su punto (x, y) . Como esa posición hereda dos incertidumbres (la de la pose y la del sensor), se propaga cada una con la derivada de esa función inversa: G_R respecto a la pose y G_z respecto a la medición. De ahí los dos sumandos de $P_{LL} = G_R P_{RR} G_R^\top + G_z R G_z^\top$, más la covarianza cruzada con la pose. KALMAN_LANDMARK se denomina así porque corrige la pose con la covarianza 3×3 del robot frente al mapa de rasgos, sin mantener la covarianza conjunta completa pose-mapa de un EKF-SLAM total (Fig. 6.2).

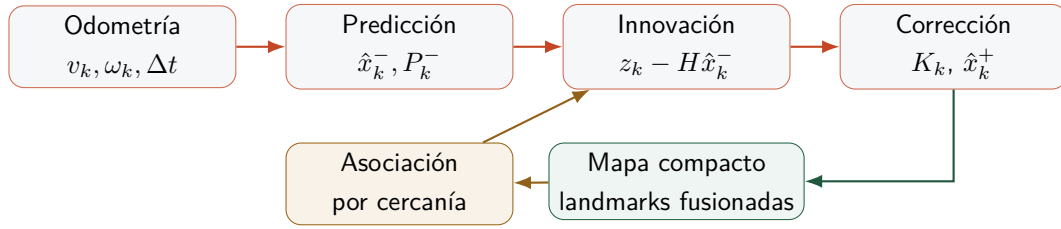


Figura 6.2: Particularidad de Kalman-landmark: corrige la pose y fusiona pocas detecciones como rasgos; no mantiene una covarianza conjunta pose-mapa como un EKF-SLAM completo.

En un entorno pequeño, esta opción mantiene estabilidad: al fusionar lecturas cercanas, el mapa no crece sin control y el planificador recibe obstáculos puntuales fáciles de filtrar. La desventaja es visual: no reconstruye superficies completas, de modo que una pared larga aparece como pocos rasgos y no como una frontera continua.

6.6 GMapping en grid

GMapping se basa en filtros de partículas Rao-Blackwellizados y mapas de grid [11]. En esta simulación se implementó una grid de ocupación con log-odds, donde $L(c) = \log(p(c)/(1 - p(c)))$. Para cada rayo, las celdas atravesadas se marcan como libres y la celda de impacto como ocupada:

$$L_t(c) = \text{clip} \left(L_{t-1}(c) + \begin{cases} l_{occ}, & c = c_{hit} \\ l_{free}, & c \in ray \end{cases} \right)$$

Los incrementos son $l_{occ} = 0,85$ y $l_{free} = -0,18$; se acotan con $\text{clip}(\cdot, -3,60, 3,60)$ para mantener las probabilidades en el rango $[0,027, 0,973]$ y evitar saturación por lecturas aisladas. Una celda se considera ocupada si $L(c) \geq 1,05$ (parámetro `gridOccupiedThreshold`).

La grid resultante se usa tanto para telemetría como para el planificador local. En Lua, el recorrido de celdas se obtiene muestreando el segmento entre el sensor y el punto detectado con paso menor que la resolución de la grid. Es una variante simple de ray-casting, suficiente para marcar las celdas libres antes de la celda de impacto (Fig. 6.3).

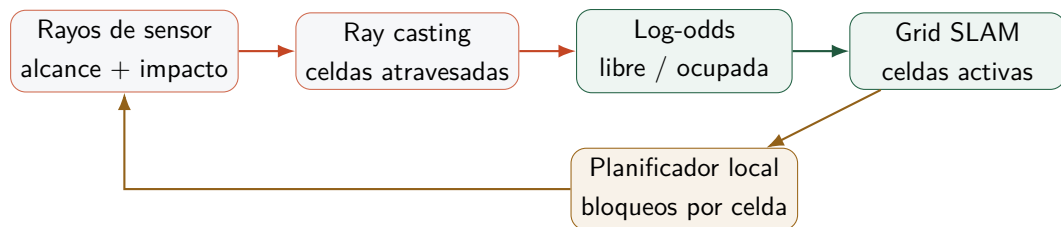


Figura 6.3: Particularidad de GMapping implementado: la información principal es una grid de ocupación. Cada rayo libera celdas antes del impacto y refuerza la celda ocupada.

El GMapping original usa un filtro de partículas Rao-Blackwellizado para mantener hipótesis de trayectoria. En esta actividad se conservó la idea que interesa para el planificador, la grid de ocupación, y se redujo el resto para que el coste computacional fuera razonable dentro del script de Coppeliasim. Esa regla explica por qué GMapping genera muchos más elementos de mapa que Kalman-landmark,

pero no necesariamente menor error de pose.

6.7 Hector con ajuste local

Hector SLAM usa *scan matching* contra una grid y depende menos de la odometría [12]. La implementación resuelve el ajuste scan-mapa mediante el método de Gauss-Newton con jacobianos analíticos, en lugar de una búsqueda exhaustiva.

La función de puntuación usa interpolación bilineal del mapa log-odds para obtener valores y gradientes continuos:

$$M_{bilin}(x, y) = \sum_{k \in \{00, 10, 01, 11\}} \lambda_k(x, y) L(c_k), \quad \nabla M_{bilin} = \frac{1}{\Delta} \begin{bmatrix} \partial_x \\ \partial_y \end{bmatrix} M_{bilin}$$

donde Δ es la resolución de la grid, λ_k son los pesos bilineales y $L(c_k)$ el valor log-odds de la celda vecina c_k . La pose óptima maximiza la puntuación media del scan:

$$\xi^* = \arg \max_{\xi} \frac{1}{N} \sum_{i=1}^N M_{bilin}(T_{\xi}(p_i)).$$

Para cada punto del scan se calcula el jacobiano analítico de la transformación rígida $T_{\xi}(p_i) = (x + r_i \cos(\theta + \beta_i), y + r_i \sin(\theta + \beta_i))$:

$$J_i = \nabla M_{bilin} \begin{bmatrix} 1 & 0 & -r_i \sin(\theta + \beta_i) \\ 0 & 1 & r_i \cos(\theta + \beta_i) \end{bmatrix} \in \mathbb{R}^{1 \times 3}$$

Gauss-Newton aproxima el Hessiano como $H \leftarrow H + J_i^T J_i$: desprecia las segundas derivadas del mapa y se queda solo con los gradientes J_i que ya calculamos, lo que basta cerca del óptimo. El paso $\Delta \xi$ sale de resolver $H \Delta \xi = b$ (con $b = \frac{1}{N} \sum_i J_i^T M_i$); como el sistema es apenas 3×3 , se resuelve con la regla de Cramer (determinantes), sin factorizaciones. Se aplican hasta 5 iteraciones, con convergencia $\|\Delta \xi\| < 0,8 \text{ mm}$ y $|\Delta \theta| < 1,5 \text{ mrad}$. Esta formulación sigue directamente la derivación de Hector SLAM [12], sin las múltiples resoluciones del paquete ROS completo (Fig. 6.4).

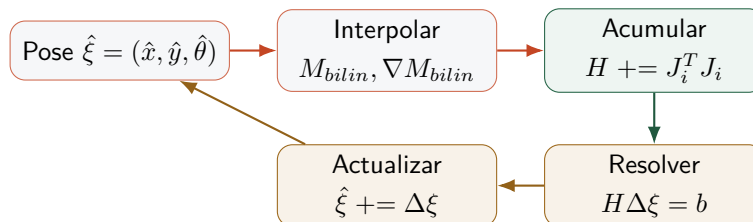


Figura 6.4: Hector SLAM: iteración Gauss-Newton con interpolación bilineal del mapa log-odds. El Hessiano se acumula con los jacobianos analíticos de la transformación rígida; el sistema $H \Delta \xi = b$ se resuelve por la regla de Cramer.

La interpolación bilineal da gradientes continuos para el Gauss-Newton, frente a una evaluación discreta de celdas. Si el entorno tiene poca textura (pasillos simétricos), el Hessiano puede ser casi

singular; la regularización diagonal 0,002 previene divergencia y el umbral de puntuación mínima (`hectorMatchMinScore`) descarta correcciones débiles.

6.8 Cartographer con submapas

Google Cartographer organiza el mapeo en submapas y cierres de ciclo para reducir deriva acumulada [13]. La implementación crea submapas por distancia recorrida (1,85 m) o periodo de tiempo (18 s), detecta cierres de ciclo cuando el robot regresa cerca de un submapa antiguo y optimiza el grafo de poses para propagar la corrección globalmente.

Un submapa se considera candidato a cierre si lleva más de 22 s creado, la distancia es inferior a 0,45 m, la separación en el grafo es de al menos tres submapas y el puntaje de scan-matching supera 0,35. Cuando se detecta un cierre válido, se registra la restricción como arista del grafo de poses:

$$e_{ij} = (\Delta x_{ij}, \Delta y_{ij}, \Delta \theta_{ij})$$

donde $(\Delta x_{ij}, \Delta y_{ij}, \Delta \theta_{ij})$ es la pose relativa medida entre el submapa activo j y el submapa antiguo i en el instante del cierre. La optimización del grafo minimiza el error cuadrático total sobre todas las restricciones almacenadas mediante descenso Gauss-Seidel:

$$\varepsilon_{ij} = ((\hat{x}_j - \hat{x}_i) - \Delta x_{ij}, (\hat{y}_j - \hat{y}_i) - \Delta y_{ij}, \text{wrap}((\hat{\theta}_j - \hat{\theta}_i) - \Delta \theta_{ij}))$$

$$\hat{x}_i += \lambda \varepsilon_{ij,x}, \quad \hat{x}_j -= \lambda \varepsilon_{ij,x}, \quad \lambda = 0,14, \quad (\text{ídem } y, \theta)$$

El residuo ε_{ij} es cuánto difiere la pose relativa estimada entre los dos submapas de la que mide el cierre. Ese error se reparte moviendo ambas poses una fracción $\lambda = 0,14$ en sentidos opuestos (una se acerca y la otra se aleja), de modo que cada paso corrige sin saltos bruscos; Gauss-Seidel es simplemente recorrer las restricciones de una en una reutilizando los valores ya actualizados. Se ejecutan 8 iteraciones por cierre detectado y la pose del robot se ancla suavemente al submapa activo tras la optimización. En la comparación de Fase 1, Cartographer generó 14 submapas y 3 cierres de ciclo. Esta formulación captura la idea central de Cartographer [13] sin el solucionador Ceres ni la optimización multi-resolución del paquete original.

El flujo de submapas y cierres de ciclo se ilustra en la Fig. 6.5.

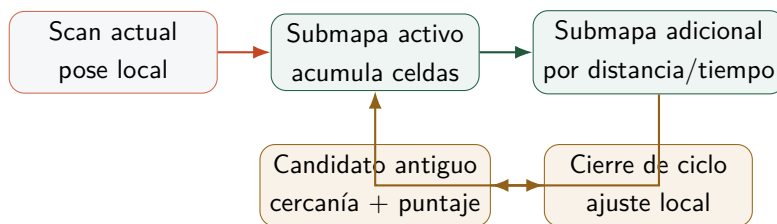


Figura 6.5: Particularidad de Cartographer-submap: el mapa no se guarda como una sola grid monolítica, sino como submapas locales con posibles cierres cuando R1 vuelve a zonas conocidas.

El mapa guarda estructura temporal: qué zona se construyó en cada tramo y los retornos a regiones ya visitadas. El coste es una lógica de navegación local más delicada: si una celda antigua queda cerca de la ruta, el planificador la trata como bloqueo; por eso en Fase 2 se añadieron filtros de persistencia, distancia al robot y recuperación directa.

6.9 Qué método usar y por qué

La Tabla 6.1 resume cuándo conviene cada método: pose y puntos cercanos a evitar (Kalman-landmark), ocupación libre/ocupada por celda (GMapping), corrección de pose por encaje del escaneo en el mapa (Hector) y submapas con reconocimiento de zonas ya visitadas (Cartographer).

Método	Conviene cuando	No conviene cuando	Lectura en esta actividad
Kalman-landmark	Se necesita pose estable y pocos obstáculos puntuales.	Se requiere una pared o contorno continuo del entorno.	Menor duración y mapa compacto; opción base para la celda compacta.
GMapping/log-odds	El planificador necesita saber qué celdas están ocupadas.	Se espera estimación completa de trayectoria por partículas.	Útil como grid de ocupación ligera; no implementa el GMapping completo.
Hector/scan-to-map	Hay suficiente geometría para alinear el scan contra el mapa.	Los rayos son pocos, simétricos o con impactos pobres.	Da buena precisión de rasgos en Fase 2, pero puede oscilar si el scan no discrimina candidatos.
Cartographer	El robot recorre zonas amplias y vuelve a regiones ya vistas.	La escena es pequeña y el coste de gestionar submapas supera el beneficio.	Logra mayor cobertura y cierres, pero tarda más y exige filtros para no bloquear la ruta.

Cuadro 6.1: Criterio de selección entre las variantes SLAM implementadas.

Con los datos de la actividad, la elección operativa es Kalman-landmark para controlar el Pioneer en esta celda. Las grids y submapas se mantienen para comparar cómo cambia el mapa cuando se prioriza ocupación, ajuste local o estructura temporal.

7

Resultados experimentales

Los resultados proceden de ejecuciones de CoppeliaSim lanzadas desde Python. Cada ejecución exporta señales a CSV y JSON, y de ahí salen las tablas y figuras. Las capturas renderizadas desde CoppeliaSim acompañan a las gráficas.

7.1 Validación funcional de la escena

La validación principal de `Sim_T2_Phase1_Basic.ttt` terminó con `passed=true`. Los indicadores más relevantes fueron:

Indicador	Resultado
Estado final de tarea	CHARGING
Modo final de movimiento	ARRIVED_TARGET
Tareas completadas	4/4
Distancia caminada por Bill	10,78 m ^a
Batería final R1	64,7 % ^a
Sensores activos	16
Rayos de sensor visualizados	16
Landmarks/celdas SLAM finales	6 (Kalman-landmark básico) ^a
Actualizaciones SLAM	2345 ^a
Error final de pose	0,040 m ^a
Error máximo de pose	0,254 m ^a
Distancia mínima positiva a obstáculo	0,215 m
Riesgo máximo de obstáculo	0,40

^a Valor extraído de `Sim_T2_Phase1_Basic_validation.json`. El comparativo SLAM extendido usa configuración distinta (más actuali

Junto con las métricas, se comprobó la ejecución en la escena: Bill recorre WS1–WS2 tomando, trabajando y entregando piezas; las estanterías, mesas y sofá están presentes; T1 y T2 son reconocibles; la cola de tareas queda registrada; y R1 vuelve a carga al terminar. La FSM de tarea recorre 19 cadenas de estado distintas durante la misión completa (listadas en `task_states_seen` del JSON de validación); el diagrama de la presentación muestra los 9 estados de tarea principales, omitiendo los estados intermedios de navegación (`PICK_RETURN_T1_WS1`, `WAIT_B1_WS2_REQUEST_T2`, `TO_WS1_DELIVER_T1`, etc.).

7.2 Desempeño del estimador Kalman-landmark

En la exportación específica del estimador, la escena usa el modo PID como base y KALMAN_LANDMARK como algoritmo SLAM. Ese modo PID de misión usa términos proporcional y derivado sobre el error de distancia, sin integral explícito; la comparación con integral completa (P/PI/PID/LQR/NMPC) se hizo en la escena de seguimiento. Los resultados de esta exportación fueron¹:

- RMSE de posición: 0,06618 m.
- Error medio de posición: 0,04496 m.
- Error P95: 0,14488 m.
- Error máximo: 0,29105 m.
- RMSE de orientación: 0,03445 rad.
- Covarianza media estimada: 0,02829.
- Landmarks finales: 8.
- Actualizaciones Kalman/SLAM: 2461.

El número de elementos del mapa depende de la representación. En Kalman-landmark son rasgos puntuales fusionados; en las variantes de grid son celdas ocupadas o grupos de celdas. La validación funcional cerró con 6 rasgos y la exportación específica Kalman-landmark con 8, ambas listas compactas, mientras que las grids acabaron con mapas más densos (26–27 celdas, Fig. 7.1).

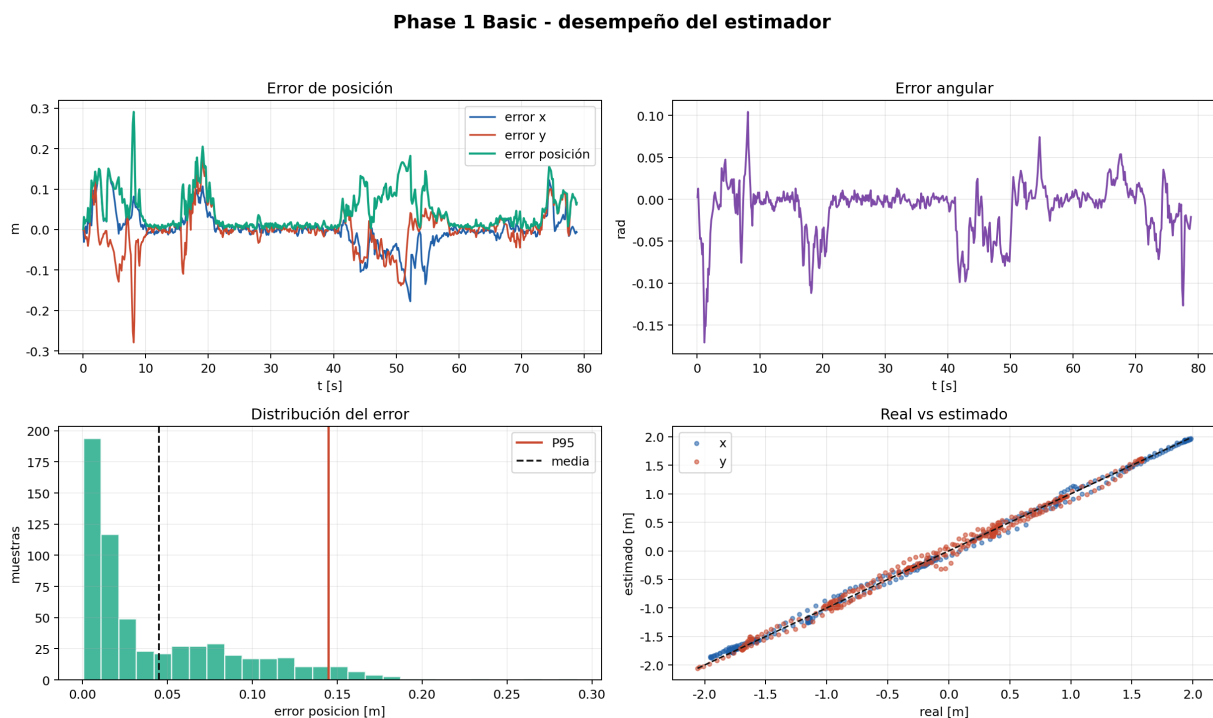


Figura 7.1: Resultados del estimador Kalman-landmark.

¹Los errores de pose difieren de la Tabla 7.2 porque ambas exportaciones usan distinta configuración de registro.

7.3 Comparación de algoritmos SLAM

La comparación usa el mismo escenario, sensores, tareas y controlador para los cuatro modos de SLAM. Todas las ejecuciones completan las 4 tareas y consumen una batería similar ($\approx 22,5\text{--}24,2\%$, coherente con la validación funcional). Los recuentos de actualizaciones SLAM ($\approx 2300\text{--}2700$) difieren ligeramente entre modos según cuántas detecciones asocia cada representación. La Tabla 7.2 recoge los valores principales.

Los tiempos y longitudes de ruta salen parecidos porque el plan de misión, los puntos de entrega y el controlador de movimiento son los mismos. Cambia la representación del mapa usada por los bloqueos y los puntos intermedios. La columna de rasgos requiere cuidado: Kalman almacena landmarks fusionadas, mientras que las variantes de grid almacenan celdas o grupos de celdas ocupadas.

Cuadro 7.2: Comparación SLAM en CoppeliaSim.

Métrica	GMapping	Hector	Cartographer	Kalman
Completado	Sí	Sí	Sí	Sí
Duración [s]	84,30	80,00	80,70	79,90
Ruta [m]	16,340	15,884	15,919	16,379
RMSE pos. [m]	0,01696	0,09079	0,07882	0,06089
Error P95 [m]	0,03097	0,24500	0,19774	0,13648
Error max. [m]	0,03896	0,28226	0,31163	0,20768
Rasgos finales	26	27	27	9
Actualizaciones SLAM	2583	2307	2382	2340
Submapas	0	0	16	0
Cierres de ciclo	0	0	2	0
Batería usada [%]	24,05	23,26	23,27	23,88

$n=1$ ejecución por método. Los resultados son indicadores cualitativos; no constituyen comparación estadísticamente validada.

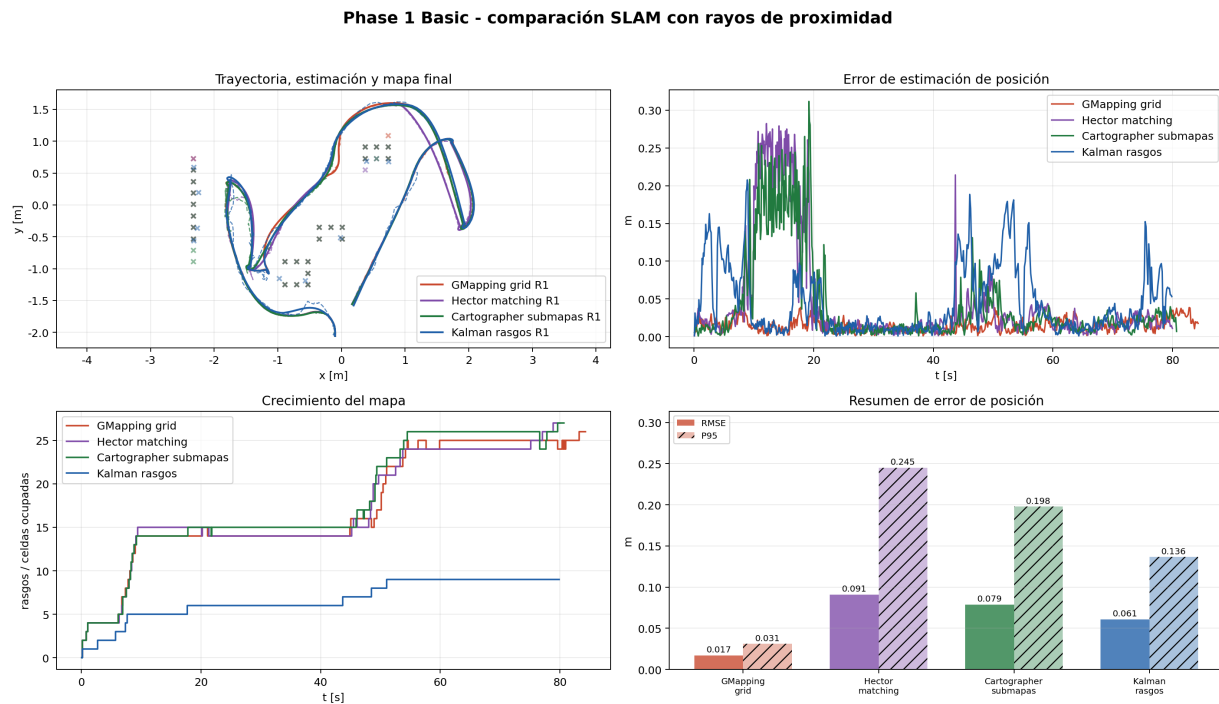


Figura 7.2: Comparación gráfica de los cuatro modos SLAM: trayectoria, precisión, crecimiento del mapa y resumen de error.

7.4 Interpretación de resultados

En este ensayo (Fig. 7.2) el menor RMSE de pose fue de GMapping (0,017 m): al apoyarse en la grid de ocupación y en el ancla global, su estimación resulta la más suave. Las variantes con corrección activa por sensor quedan por encima: Kalman-landmark 0,061 m (EKF de rango y rumbo), Cartographer 0,079 m y Hector 0,091 m (ajuste *scan-to-map* Gauss-Newton); inyectar la información ruidosa de los 16 ultrasonidos en la pose tiene ese coste. Kalman-landmark fue el más rápido (79,9 s) y dejó el mapa más compacto (9 rasgos); Hector recortó la ruta a 15,884 m con el menor consumo de batería (23,26 %); Cartographer registró estructura temporal con 16 submapas y 2 cierres de ciclo. Todas completaron la misión.

La escena base usa KALMAN_LANDMARK como estimador aunque no tenga el menor RMSE: entrega al planificador una lista compacta de obstáculos puntuales fáciles de filtrar. Las grids y submapas se conservan para comparar cómo cambia el mapa con los mismos rayos de proximidad.

8

Fase 2: SLAM con mapa desconocido

La escena `Sim_T2_Phase2_SLAM_Unknown.ttt` parte de la celda validada y arranca con parte del entorno oculto. R1 ve cajas, paredes y un segundo robot móvil autónomo a través de los sensores de proximidad dibujados como rayos. El controlador actualiza el mapa construido hasta el momento y lo usa para activar puntos intermedios locales.

El obstáculo dinámico es un robot móvil (`/P2_Wanderer`) que recorre la zona central de la celda siguiendo *rutasy aleatorias*: el gestor `Phase2_Unknown_Map_Manager` muestrea waypoints al azar (con semilla fija para que la ejecución sea reproducible) y lo conduce de forma cinemática hacia cada uno. R1 lo detecta con su anillo de 16 sensores y lo evita de forma reactiva, pero no lo añade al mapa SLAM (es un agente móvil, como Bill); así, un objeto en movimiento no entra en el mapa. Las cajas y los marcadores de zona desconocida quedan fijos como obstáculos no modelados.

Como el mapa empieza vacío, la misión arranca con una *pasada exploratoria*: antes de intentar ninguna tarea, R1 recorre en orden de cercanía una serie de *frontiers* repartidas por el área de operación (estado interno `EXPLORE_FRONTIERS`), de modo que el anillo de ultrasonidos barre la celda y el mapa SLAM se rellena. Es una marcha deliberadamente lenta y cautelosa, sin manipular piezas, acotada en el tiempo: cuando alcanza las *frontiers* o agota su ventana, comienza el trabajo sobre el mapa que acaba de construir. Las *frontiers* se sitúan en los pasillos despejados centrales para que siempre sean alcanzables; el barrido es del área operativa, no de los rincones pegados a las paredes.

Tras explorar, R1 *repite el ciclo de trabajo varias veces* sobre la misma celda en lugar de detenerse tras la primera pasada. El mapa SLAM y el conocimiento acumulado se conservan entre vueltas (Tabla 8.1): cada repetición reutiliza un mapa más completo. La velocidad de avance se gobierna además con un factor de confianza que crece con el número de vueltas completadas y con las actualizaciones SLAM acumuladas¹. Para que el ciclo no se bloquee nunca, la navegación incorpora varias salvaguardas: el robot retrocede un instante al salir de una estantería; si una maniobra se estanca (por el robot móvil o un hueco estrecho) ejecuta un arco de retroceso para reorientarse; y una geo-valla reincorpora el chasis al recinto si un artefacto del motor de física llegara a expulsarlo.

La Tabla 8.1 mide por fase la pasada exploratoria y las tres vueltas de trabajo, promediando tres ejecuciones. Conviene leerla con honestidad. Lo que mejora de forma *monótona* es el conocimiento del entorno: la pasada exploratoria construye el mapa inicial (de 0 a $\approx 2\,600$ actualizaciones SLAM y 5 *landmarks*) sin tocar ninguna pieza, y a partir de ahí el mapa se densifica vuelta a vuelta ($\approx 5\,500 \rightarrow 8\,500 \rightarrow 10\,800$ actualizaciones; 9–10 *landmarks* estables). El recorrido por vuelta es casi constante (≈ 23 m) porque la ruta de tareas es la misma. En cambio, la *velocidad media medida no crece de forma limpia* entre vueltas: la domina la interacción con el robot móvil, que cambia en cada

¹Señal interna `phase1SpeedLearnFactor`; pasa de $\approx 0,60$ en la primera vuelta a $\approx 0,88$ en la tercera, acotado por debajo de 1 para conservar margen de seguridad al maniobrar junto a las estanterías.

ejecución y llega a frenar más la tercera vuelta. Es decir, el factor de confianza interno sí escala con el mapa, pero su efecto sobre la velocidad media queda enmascarado por el obstáculo dinámico y por los tiempos fijos (Bill trabajando, manipulación, recarga). Ninguna de las tres ejecuciones necesitó maniobras de recuperación, lo que indica que la navegación es estable en las tres vueltas.

Cuadro 8.1: Métricas de navegación por fase en la celda desconocida (media de 3 ejecuciones). “Explorar” es la pasada inicial de mapeo; las vueltas 1–3 son los ciclos de trabajo sobre el mapa acumulado.

Fase	Dur. [s]	Recorr. [m]	Vel. media [m/s]	Updates SLAM	Recup.
Explorar	43,7	6,6	0,155	2 562	0
Vuelta 1	117,8	23,4	0,197	5 456	0
Vuelta 2	123,4	23,6	0,193	8 480	0
Vuelta 3	145,0	23,1	0,162	10 819	0

Updates SLAM es el valor acumulado al cerrar cada fase; *Recup.* cuenta maniobras de des-atasco.

8.0.1 ¿Mejora el SLAM con cada vuelta?

La velocidad media no es el indicador adecuado de aprendizaje (la enmascara el robot móvil). Lo correcto es mirar la *calidad del propio SLAM*: si el mapa *converge* y si la *localización* se afina al densificarse. La Tabla 8.2 lo mide por fase, promediando tres ejecuciones, y la respuesta es afirmativa con dos firmas claras:

- **El mapa converge.** Los *landmarks* nuevos descubiertos por fase caen de ≈ 4 durante la exploración y la primera vuelta a ≈ 1 en la segunda y la tercera: el robot deja de encontrar estructura nueva porque ya ha mapeado la celda (el número total de *landmarks* se estabiliza en torno a 10–11). Es la firma de convergencia de un SLAM.
- **La localización se afina.** El error de pose medio baja de $\approx 0,086$ m mientras explora un mapa casi vacío a $\approx 0,03$ m una vez construido el mapa, y se mantiene ahí: con más *landmarks* fusionadas, el EKF dispone de más referencias para corregir la pose. Es decir, la mejora grande llega al pasar de “sin mapa” a “con mapa” (exploración $\rightarrow$ vuelta 1), y luego el sistema queda convergido y estable, no mejorando indefinidamente.

Cuadro 8.2: Calidad del SLAM por fase en la celda desconocida (media de 3 ejecuciones): convergencia del mapa y precisión de localización.

Fase	<i>Landmarks</i> (fin)	Nuevas	Error pose medio [m]	Error pose máx. [m]
Explorar	4	4	0,086	0,32
Vuelta 1	9	4	0,033	0,27
Vuelta 2	10	1	0,042	0,33
Vuelta 3	11	1	0,031	0,26

Nuevas = *landmarks* descubiertas en esa fase (la caída a ≈ 1 indica convergencia del mapa).

La métrica de actividad de mapa en Fase 2² cuenta rasgos detectados, celdas ocupadas y actualizaciones SLAM acumuladas durante la misión. Los métodos de grid la acumulan mucho más rápido que Kalman-landmark, porque cada rayo marca muchas celdas desde el primer ciclo mientras que Kalman fusiona pocas landmarks discretas, y terminan en 90–91 % frente al 50 % de Kalman. Como una mayor actividad no implica mejor geometría, la cobertura contra la referencia se revisa aparte con la exhaustividad de obstáculos y la precisión de rasgos.



(a) Inicio con zona desconocida.

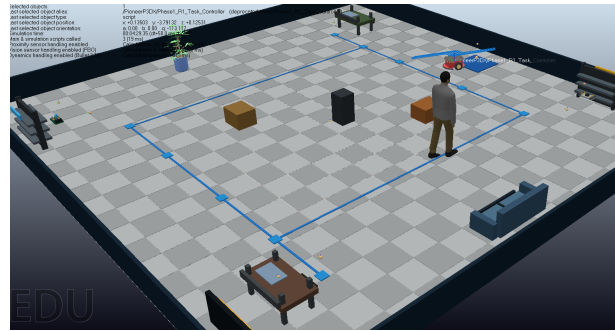


(b) Replanificación ante el robot móvil dinámico.

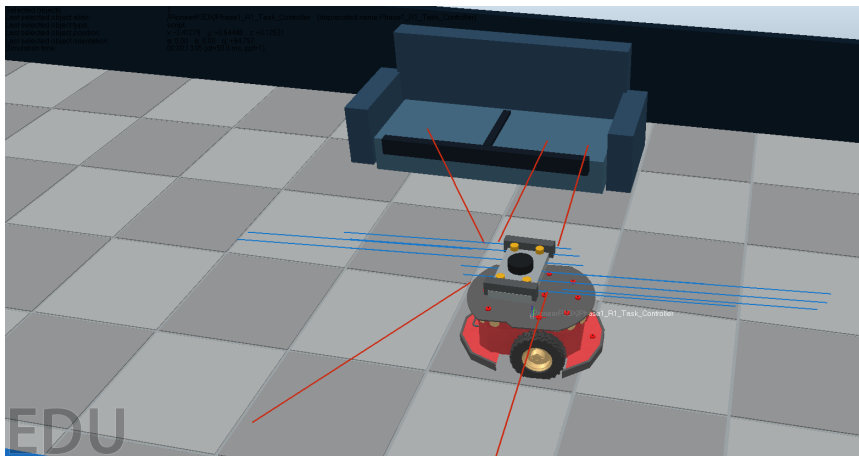
Figura 8.1: Capturas 3D de la escena Fase 2 ejecutada en CoppeliaSim.

Las capturas operativas se recogen en la Fig. 8.1; el conjunto extendido con lecturas de sensores y plano general de la celda se muestra en la Fig. 8.2.

²Señal interna `phase2MappingEvidencePct`.



(a) Zona desconocida, Bill y primer bloque no modelado. (b) Plano general de la ruta y los elementos de la celda.



(c) Lecturas de proximidad frente al sofá.

Figura 8.2: Capturas de ejecución de Fase 2 con mapa desconocido y obstáculos en la celda.

8.1 Recuperación ante bloqueo local

En Fase 2, las variantes de grid podían quedar atrapadas en estados de recogida, con alternancia prolongada entre AVOIDING y SLAM_WAYPOINT. El planificador trataba como bloqueo cualquier celda ocupada cerca del segmento al objetivo, incluso cuando la celda estaba pegada al robot, a la estantería o al punto de recogida.

El planificador aplica cinco reglas para evitar ese ciclo:

- Las celdas de grid necesitan varias detecciones antes de influir en la planificación.
- Los obstáculos muy cercanos al robot o al objetivo no fuerzan un punto intermedio.
- Los candidatos de desvío reciben penalización si salen del área útil del warehouse.
- Tras alcanzar un punto intermedio, R1 mantiene una ventana breve de ruta directa para evitar replanificar contra el mismo obstáculo.
- Si una variante de grid se queda varios segundos sin reducir distancia al objetivo, se activa RECOVERY_DIRECT: la planificación ignora el mapa durante una ventana corta y conserva la evitación reactiva.

Con estas reglas los cuatro modos completan la misión en Fase 2. En la ejecución final, Cartographer activó una recuperación del planificador; GMapping, Hector y Kalman terminaron sin usarla.

8.2 Comparación de métodos

Los cuatro métodos afrontan el mismo escenario de Fase 2: mapa incompleto al inicio, obstáculos no modelados y un robot móvil cruzando la celda. Aquí interesa qué representación sostiene la misión sin bloqueo, más que la cantidad de mapa acumulada:

Kalman-landmark: pose estable con pocos rasgos. **GMapping:** ocupación por celda. **Hector:** pose corregida por ajuste scan-grid. **Cartographer:** submapas y cierres de ciclo, a costa de más condiciones para decidir si una celda antigua bloquea la ruta.

Cuadro 8.3: Comparación Fase 2 con mapa desconocido y un robot móvil dinámico.

Métrica	GMapping	Hector	Cartographer	Kalman
Completado	Sí	Sí	Sí	Sí
Duración [s]	92,45	91,55	104,20	88,60
Ruta [m]	15,759	15,666	16,711	16,407
RMSE pos. [m]	0,01664	0,08182	0,04011	0,07392
Error P95 [m]	0,02186	0,19962	0,08094	0,13366
Actividad de mapa [%]	90,421	90,433	91,288	50,119
Rasgos finales	29	31	27	9
Exhaustividad aprox. final	0,500	0,500	0,500	0,500
Precisión aprox. final	0,172	0,323	0,333	0,333
Replanificaciones	15	16	20	14
Recuperaciones del planificador	0	0	0	0
Batería usada [%]	23,414	23,114	25,466	24,334
Submapas	0	0	16	0
Cierres de ciclo	0	0	4	0

Cartographer registró 16 submapas y 2 cierres de ciclo en Fase 1 y 16 submapas y 4 cierres en Fase 2. Los cierres se computan en tiempo de ejecución según la proximidad del robot a submapas antiguos, por lo que las diferencias entre fases provienen de las trayectorias y detecciones distintas de cada escena, no de un valor prefijado.

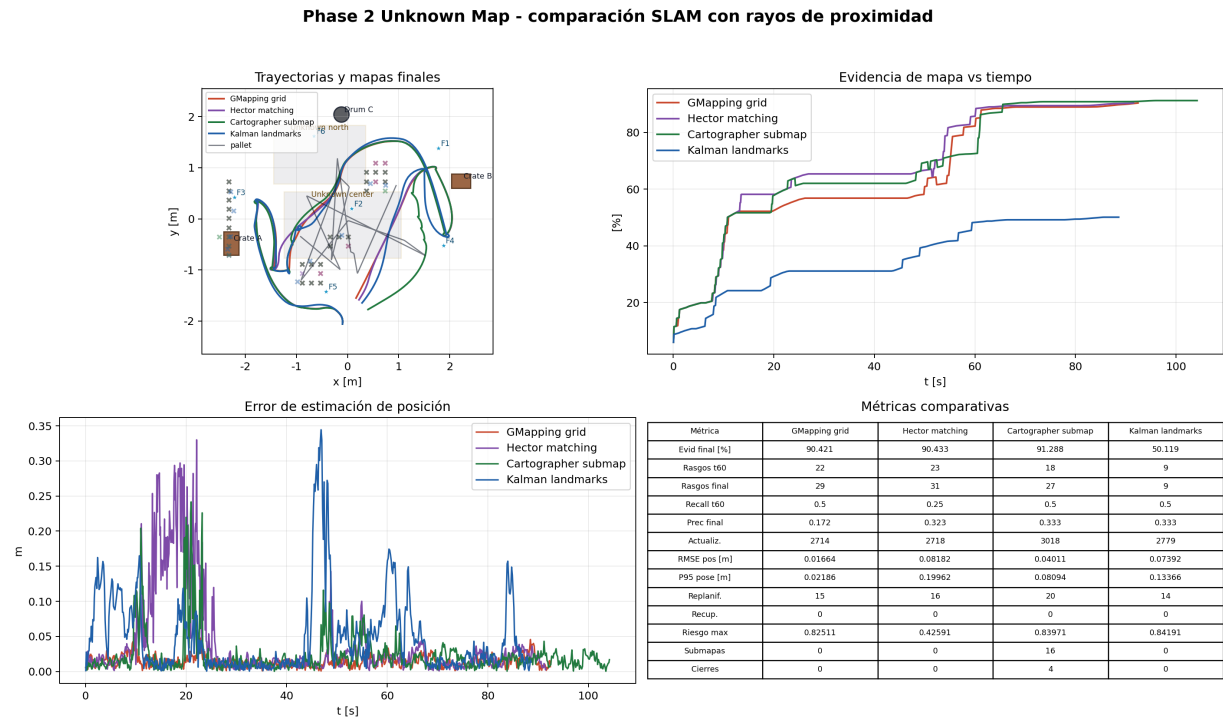


Figura 8.3: Comparación de métodos en Fase 2.

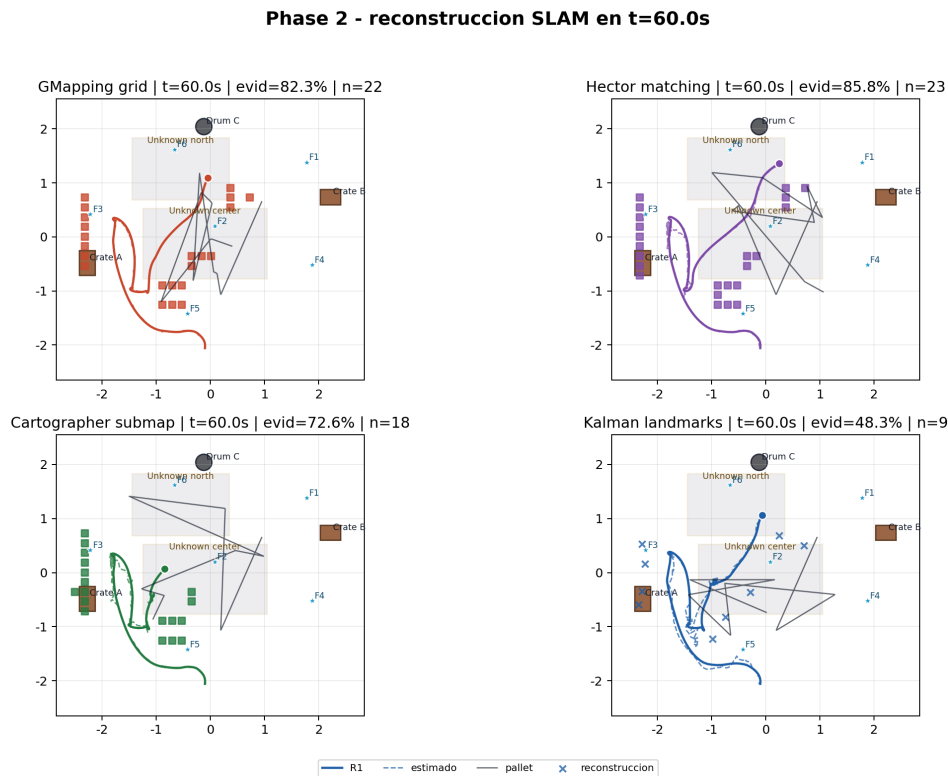


Figura 8.4: Reconstrucción del mapa al mismo instante común, $t = 60$ s.

8.3 Discusión técnica

La Tabla 8.3 recoge los valores numéricos completos; las Figs. 8.3 y 8.4 los resumen gráficamente junto con la reconstrucción del mapa. Kalman-landmark terminó la misión en el menor tiempo (88,6 s) con el mapa más compacto (9 rasgos), aunque su RMSE de pose (0,074 m) no fue el menor: la corrección EKF a partir de 16 ultrasonidos dispersos introduce más ruido que apoyarse en la grid. GMapping obtuvo el menor RMSE (0,017 m) por esa misma razón.

Los modos de grid generan más rasgos de mapa y aumentan rápido la actividad de mapeo, métrica que cuantifica actividad y no calidad geométrica. Hector hizo la ruta más corta (15,666 m) con el menor consumo de batería (23,1 %), pero el mayor RMSE (0,082 m), coherente con la sensibilidad del *scan matching* cuando los rayos discriminan poco. Cartographer fue el más caro (104,2 s y 25,5 % de batería) porque gestiona submapas, pero a cambio aportó estructura temporal (16 submapas, 4 cierres de ciclo) y el menor RMSE entre las grids con scan matching (0,040 m). Ningún modo necesitó RECOVERY_DIRECT en esta ejecución.

Para la misión, Kalman-landmark queda como base por su mapa compacto y bajo coste de cómputo. Para representar ocupación, las grids generan mapas más densos, pero su utilidad debe evaluarse junto con error de pose, ruta, replanificaciones y recuperaciones.

9

Validación

Se revisan el comportamiento del sistema y los archivos generados. La lista de verificación (tabla siguiente) recorre escena, objetos, scripts, tareas, movimiento de Bill, acciones visuales, anticolisión, error del estimador y rayos de sensor. Las capturas 3D se obtuvieron desde CoppeliaSim con `capture_coppelasim_screenshots.lua`; son las imágenes de las Figuras 2.2 y 8.1.

9.1 Validación por lista de verificación

Bloque	Verificación	Estado
Escena	Existe <code>Sim_T2_Phase1_Basic.ttt</code> y contiene R1, B1, T1, T2, C1.	OK
Mannequin	B1 (Bill) se usa como persona móvil tipo <code>People/mannequin</code> .	OK
Warehouse	Dos mesas, dos estanterías, sofá, obstáculos y panel de tareas.	OK
Scripts	Controlador R1 y manager B1 adjuntos en la escena.	OK
Tareas	Entrega y retorno de T1 y T2; cuatro tareas completadas.	OK
Bill	Movimiento entre estaciones y acciones de tomar/trabajar/entregar.	OK
Percepción	16 sensores activos y rayos visualizados.	OK
Seguridad	Modo <code>AVOIDING</code> observado y señal de riesgo activa.	OK
Estimación	Kalman activo, mapa SLAM actualizado y error acotado.	OK
Planificación	SLAM_WAYPOINT observado.	OK
Batería	Tarea aceptada por batería y retorno a carga.	OK
Capturas CoppeliaSim	Vista general, entrega, trabajo de B1, rayos de sensores y replanificación Fase 2.	OK
Fase 2	Mapa desconocido, robot móvil con rutas aleatorias, cuatro SLAM completados y recuperación del planificador registrada.	OK
Deck editable	Existe <code>slides/Actividad2_Pioneer_CoppeliaSim.pptx</code> junto con su PDF exportado.	OK

9.2 Timeline de la misión

La Fig. 9.1 muestra la secuencia temporal exportada con los estados de tarea y movimiento durante la misión.

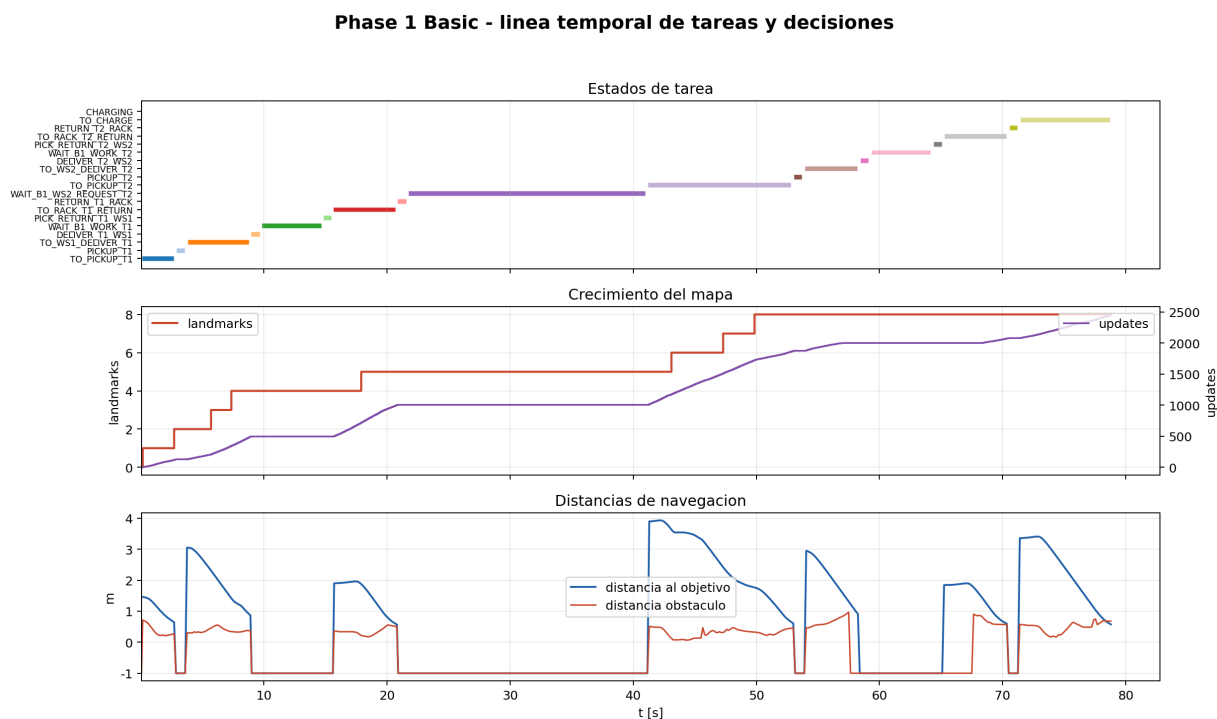


Figura 9.1: Secuencia temporal de la simulación: estados de tarea, movimiento, planificador y acciones de Bill durante la misión.

La misión incluye esperas de proceso. R1 se detiene mientras Bill trabaja T1 o T2 y espera a que B1 llegue a WS2 antes de buscar T2. Esa espera aparece como WAIT_B1; sin ese estado, la escena presenta movimiento sin respetar la secuencia lógica de entrega.

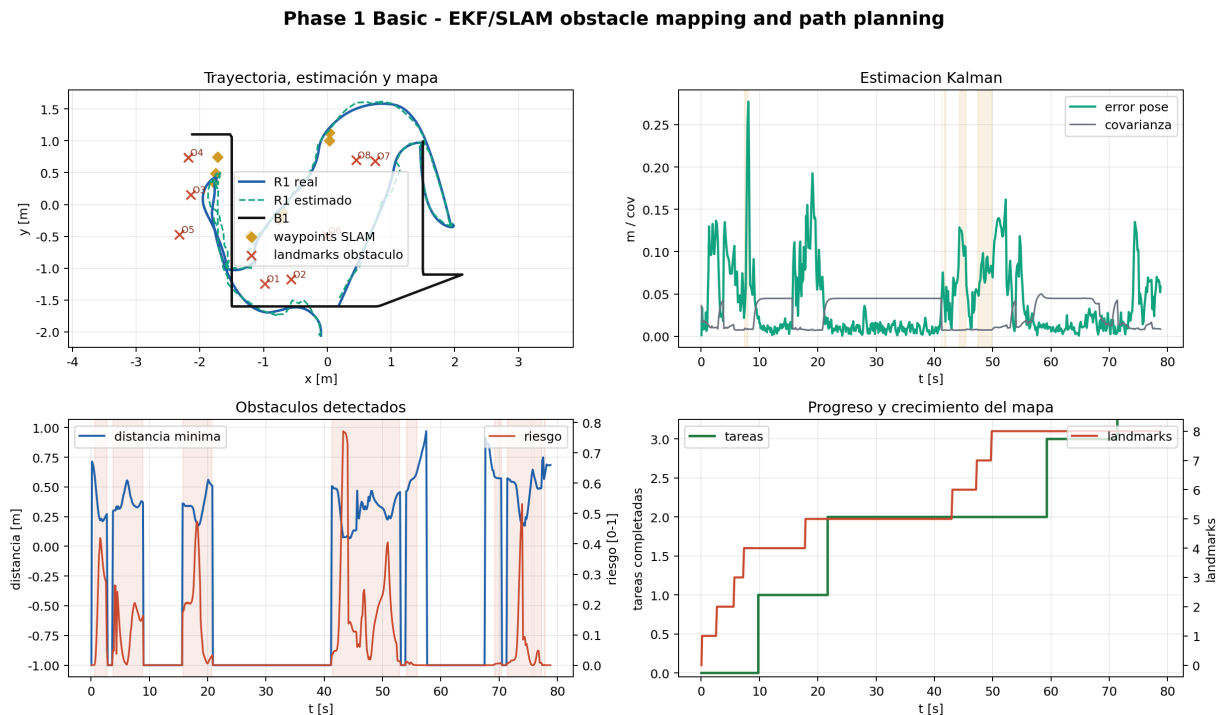


Figura 9.2: Panel de la ejecución base: pose, error, batería, obstáculos, planificador y estados principales.

El panel completo de la ejecución base queda recogido en la Fig. 9.2.

9.3 Incidencias corregidas

Durante la validación se corrigieron tres incidencias.

9.3.1 Sincronización de tarea T2

R1 podía iniciar T2 antes de que Bill llegara a WS2, por lo que la tarea no representaba una entrega real. La máquina de estados no comprobaba la posición de Bill al pasar de devolver T1 a recoger T2: la transición ocurría en cuanto R1 terminaba la tarea anterior. Se corrigió bloqueando el estado de solicitud de T2 hasta que Bill llega a WS2.

9.3.2 Ciclos del planificador en Fase 2

Algunas celdas de grid persistentes quedaban demasiado cerca del objetivo y el planificador entraba en un ciclo de desvíos cortos. Las representaciones de grid acumulan evidencias pasadas que no se desvanecen, de modo que cuando un obstáculo desaparece o el objetivo cae dentro del radio de celdas marcadas, la planificación oscila. El parche exige detecciones persistentes, ignora celdas pegadas al objetivo y vuelve a ruta directa cuando no hay progreso. Es un límite conocido de los planificadores sobre grids de ocupación sin olvido temporal.

9.3.3 Oscilación del scan matching tipo Hector

Con pocos impactos de sensor, varios candidatos de pose recibían puntajes parecidos y la corrección oscilaba. En entornos con poca textura (pasillos rectos) el scan matching produce un paisaje de puntuación casi plano, donde mínimas diferencias de ruido invierten el candidato ganador de un ciclo a otro; es un modo de fallo conocido del ajuste de scan en geometrías poco informativas [10]. Se mitigó con una ventana de búsqueda pequeña, un umbral mínimo de puntuación y una ganancia de corrección parcial.

Conclusiones

La escena base comprueba el ciclo completo entre R1, Bill, T1 y T2: el robot entrega cada herramienta, espera a que Bill trabaje y recoge la pieza, evitando obstáculos, hasta volver a la estación de carga.

En Fase 2, Kalman-landmark resultó el modo operativo para esta celda compacta: no logró el menor RMSE (lo hizo GMapping, con 0,017 m), pero terminó en 88,6 s con la lista de obstáculos más compacta, fácil de filtrar por el planificador. GMapping, Hector y Cartographer comparan tres representaciones de mapa (grid log-odds, scan-matching Gauss-Newton y submapas con cierre local) con las mismas lecturas de proximidad y el mismo PID. Cada modo se ejecutó una vez ($n = 1$), de modo que la ordenación es indicativa y puede cambiar con otras condiciones de inicialización o trayectorias de Bill.

El anticolisiones funciona como capa reactiva: cuando los sensores detectan algo cerca, el robot reduce velocidad y corrige el giro, y actualiza la señal de riesgo. La capa de punto intermedio SLAM filtra celdas de grid poco observadas, ignora obstáculos pegados al objetivo y penaliza desvíos fuera del área útil. En los mapas de grid se añadió una recuperación por falta de progreso, con ruta directa durante una ventana corta y evitación reactiva siempre activa.

La comparación entre P, PI, PID, LQR y NMPC se conserva como análisis complementario. En el seguidor de Bill, PID dio el menor error final (0,0457 m), LQR el menor error medio durante el movimiento (0,0167 m, seguido de NMPC con 0,0189 m) y NMPC la mayor suavidad de comando (RMS de Δu de 0,0457 y solo 3 inversiones de sentido). En la misión T1 y T2, PI obtuvo el mejor puntaje ponderado de los CSV disponibles. El LQR usa una ganancia de Riccati calculada en escena y el NMPC un control predictivo de horizonte completo resuelto con una búsqueda de patrón de Hooke-Jeeves.

Limitaciones y trabajo futuro

La comparación de controladores se realizó con una única carrera por modo ($n = 1$) y sin análisis de estabilidad formal ante saturaciones, conmutación entre modos o cambios dinámicos de objetivo. La validación es experimental: las ejecuciones completan la misión sin divergencia observable, pero una prueba formal requeriría análisis de Lyapunov o estabilidad de modos conmutados. Las variantes de mapeo tampoco se evaluaron contra una referencia geométrica absoluta: el RMSE se mide respecto a la pose ruidosa del simulador, y las métricas de cobertura son proxies de actividad del mapa, no mediciones de IoU (*Intersection over Union*) contra un mapa de referencia.

Además de la bibliografía académica, se consultaron repositorios públicos para contrastar estructuras de implementación: `slam_gmapping`, `hector_slam`, Cartographer y PythonRobotics [14, 15, 16, 17]. Gemini y Claude se usaron como apoyo de consulta e ideación visual; el código final, las métricas, las figuras y la redacción técnica fueron revisados y adaptados manualmente [18].

Como trabajo futuro, el sistema podría conectarse a ROS 2 para ejecutar los paquetes originales de `slam_toolbox`, Cartographer, Hector o GMapping y comparar así las implementaciones Lua con sus homólogos completos. La evaluación de mapas debe incorporar referencia geométrica, IoU de ocupación,

precisión y exhaustividad por obstáculo y varias ejecuciones por método para obtener significancia estadística.

Referencias

- [1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. MIT Press, 2 ed., 2011.
- [2] S. G. Tzafestas, *Introduction to Mobile Robot Control*. Elsevier, 2014.
- [3] MobileRobots Inc., *Pioneer 3 Operations Manual*, 2006. Manual de operación para plataformas Pioneer 3.
- [4] R. Hooke and T. A. Jeeves, ““direct search” solution of numerical and statistical problems,” *Journal of the ACM*, vol. 8, no. 2, pp. 212–229, 1961.
- [5] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*. Prentice Hall, 1990.
- [6] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 ed., 2017.
- [7] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.
- [8] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [9] V. Braitenberg, *Vehicles: Experiments in Synthetic Psychology*. MIT Press, 1984.
- [10] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [11] G. Grisetti, C. Stachniss, and W. Burgard, “Improved techniques for grid mapping with rao-blackwellized particle filters,” *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [12] S. Kohlbrecher, O. von Stryk, J. Meyer, and U. Klingauf, “A flexible and scalable slam system with full 3d motion estimation,” in *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics*, pp. 155–160, 2011.
- [13] W. Hess, D. Kohler, H. Rapp, and D. Andor, “Real-time loop closure in 2d lidar slam,” in *2016 IEEE International Conference on Robotics and Automation*, pp. 1271–1278, 2016.
- [14] ROS Perception, “slam_gmapping,” 2018. Consultado en mayo de 2026. Repositorio público: https://github.com/ros-perception/slam_gmapping.
- [15] TU Darmstadt ROS Package Repository, “hector_slam,” 2014. Consultado en mayo de 2026. Repositorio público: https://github.com/tu-darmstadt-ros-pkg/hector_slam.
- [16] Cartographer Project, “Cartographer,” 2016. Consultado en mayo de 2026. Repositorio público: <https://github.com/cartographer-project/cartographer>.

- [17] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics," 2018. Consultado en mayo de 2026. Referencia de algoritmos de robótica móvil: <https://github.com/AtsushiSakai/PythonRobotics>.
- [18] Google, "Gemini," 2026. Herramienta usada como apoyo de consulta e ideación visual; no usada como fuente primaria de validación técnica.



Anexos

A.1 Archivos principales

Archivo	Función
coppeliasim/Sim_T2_Phase1_Basic.ttt	Escena final validada.
coppeliasim/Sim_T2_Phase1_Basic_validation.json	Resultado de validación automática.
coppeliasim/build_phase1_basic_scene.py	Generador de escena y validador.
coppeliasim/phase1_r1_task_controller.lua	Controlador principal de R1: tareas, navegación, SLAM, batería y telemetría.
coppeliasim/phase1_b1_walking_manager.lua	Movimiento y acciones visuales de B1 (Bill).
coppeliasim/capture_coppeliasim_screenshots.lua	Capturas 3D renderizadas desde CoppeliaSim para el informe.
coppeliasim/Sim_T2_Phase2_SLAM_Unknown.ttt	Escena con mapa desconocido, obstáculos no modelados y un robot móvil autónomo (P2_Wanderer) con rutas aleatorias.
coppeliasim/build_phase2_unknown_slam_scene.py	Generador y validador de Fase 2.
coppeliasim/Actividad2_1_Basic_Follower.ttt	Escena básica de seguimiento de Bill.
coppeliasim/pioneer_pid_follower_controller.lua	Controlador del seguidor con P, PI, PID, LQR y NMPC por disparo directo.

Cuadro A.1: Escenas y controladores principales.

Archivo	Función
coppeliassim/run_phase1_slam_export.py	Exportación CSV de la ejecución base.
coppeliassim/run_phase1_slam_comparison.py	Comparación de algoritmos SLAM implementados en Lua.
coppeliassim/plot_phase1_slam_comparison.py	Generación de gráficas de comparación SLAM.
coppeliassim/phase1_slam_logs/	CSV, JSON, Markdown y figuras de la comparación SLAM.
coppeliassim/run_phase2_slam_unknown_comparison.py	Exportación comparativa Fase 2 para los cuatro modos SLAM.
coppeliassim/plot_phase2_slam_unknown_comparison.py	Figuras y métricas de reconstrucción Fase 2.
coppeliassim/phase2_slam_logs/	CSV, JSON, Markdown y figuras de Fase 2.
coppeliassim/phase1_controller_logs/	Registros CSV y gráficas de comparación de controladores.
coppeliassim/follower_pid_logs/	CSV y gráficas de seguimiento de Bill.

Cuadro A.2: Exportadores, registros y escena complementaria.

A.2 Parámetros relevantes

Parámetro	Valor	Comentario
Radio de rueda	0,0975 m	Conversión de velocidad lineal a angular de motor.
Distancia entre ruedas	0,331 m	Modelo diferencial del Pioneer 3DX.
Velocidad máxima (vMax)	0,62 m/s	Saturación de avance (controlador de misión).
Velocidad angular máxima (wMax)	1,42 rad/s	Saturación de giro (controlador de misión).
Rango de evitación	0,54 m	Umbral de influencia reactiva de obstáculos.
Parada crítica	0,10 m	Distancia frontal mínima antes de detener el avance.
Batería inicial	96 %	Valor usado en validaciones.
Umbral batería baja	56 %	Limita velocidad si se alcanza.
Umbral batería crítica	26 %	Modo conservador.
Resolución de grid	0,18 m	Usada por GMapping, Hector y Cartographer en grid.
Margen del planificador	0,40 m	Separación mínima deseada respecto al mapa.
Desplazamiento lateral	0,52 m	Distancia lateral del punto intermedio local.
Retardo de recuperación	6,0 s	Activa RECOVERY_DIRECT si no hay progreso hacia el objetivo.
Duración de recuperación	3,5 s	Ventana en la que se suspende el mapa para planificar, sin apagar la evitación local.

Cuadro A.3: Parámetros de configuración del sistema.

A.3 Fragmentos Lua propios

Listing A.1: Conversión de velocidad diferencial a ruedas.

```
function DifferentialDrive:setVelocity(v, w)
    local left = (v - 0.5 * self.cfg.trackWidth * w) / self.cfg.wheelRadius
    local right = (v + 0.5 * self.cfg.trackWidth * w) / self.cfg.wheelRadius
    sim.setJointTargetVelocity(self.handles.leftMotor, left)
    sim.setJointTargetVelocity(self.handles.rightMotor, right)
end
```

Listing A.2: Entrega, espera de trabajo humano y retorno de herramienta.

```
local function deliverToolToBill(tool, workSurface, completedCount, nextSpec,
    nextState, afterDeliver)
    stopRobot('MANIPULATING')
    if delayElapsed(cfg.manipulationDelay) then
        carryingTool = 0
        placeToolAt(tool, workSurface, 0.43)
        completedTaskCount = math.max(completedTaskCount, completedCount)
        currentTaskSpec = nextSpec
        setShapeColorSafe(tool, ToolColor.onTable)
        if afterDeliver then afterDeliver() end
        setTaskState(nextState)
    end
```

```

    end
end

local function waitForBillWork(tool, workSurface, dropPoint, workDelay,
    nextTaskId, nextSpec, nextState)
    stopRobot('WAIT_B1')
    placeToolAt(tool, workSurface, 0.43)
    if delayElapsed(workDelay) then
        currentTaskId = nextTaskId
        currentTaskSpec = nextSpec
        placeToolAt(tool, dropPoint, 0.36)
        setTaskState(nextState)
    end
end
end

```

Listing A.3: Animación de Bill trabajando con brazos hacia la mesa.

```

elseif self:actionContains(action, 'WORKING') then
    pose.leftShoulder = 0.62 + 0.06 * pulse
    pose.rightShoulder = 0.60 - 0.06 * pulse
    pose.leftElbow = 0.28 + 0.05 * math.abs(tap)
    pose.rightElbow = 0.26 + 0.05 * math.abs(tap)
    pose.leftKnee = 0.07 + 0.03 * math.abs(pulse)
    pose.rightKnee = pose.leftKnee

```

A.4 Comparación de control

Las métricas complementarias de control se ilustran en la Fig. A.1.

Modo	Dur. [s]	Mov. [s]	Ruta [m]	Bat. [%]	Riesgo max.	Puntaje
P	105,35	62,75	21,096	30,145	0,682	0,729
PI	98,70	56,50	21,880	31,126	0,836	0,810
PID	90,60	51,35	21,371	30,086	0,494	0,089
LQR	98,90	56,55	21,183	29,972	0,740	0,376
NMPC	94,60	52,10	21,387	29,919	0,718	0,202

Cuadro A.4: Resultados comparativos de los cinco controladores en la misión warehouse de Fase 1 ($n = 1$ por modo; puntaje compuesto, menor es mejor). Aquí LQR y NMPC son las variantes reactivas simplificadas del controlador de tarea; el LQR con ganancia de Riccati y el NMPC de horizonte completo se evalúan sobre el follower (Tabla 4.1).

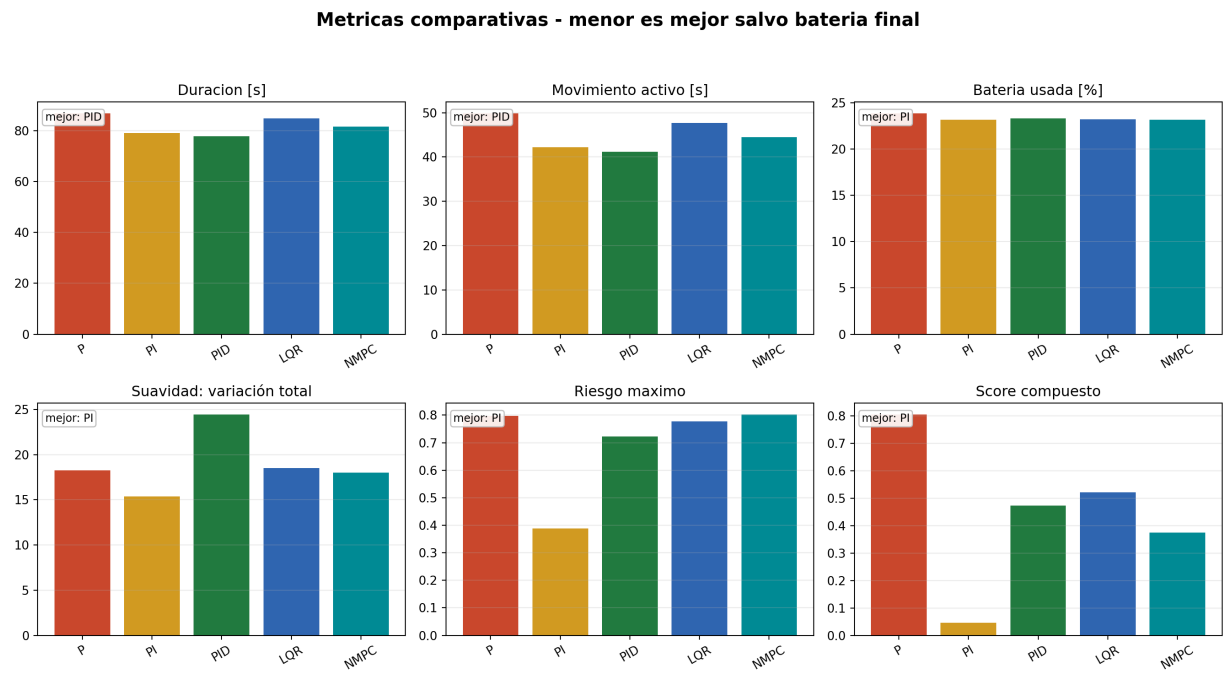


Figura A.1: Métricas complementarias de control: tiempos, trayectoria, suavidad, batería, riesgo y error de pose.

A.5 Parámetros de Cartographer

Los parámetros del módulo Cartographer-submap usados en Fase 1 y Fase 2 son (Tabla A.5):

Parámetro	Valor
cartographer.submap_min_age_s	22 s
cartographer.submap_max_distance_m	0,45 m
cartographer.loop_closure_score	0,35

Cuadro A.5: Parámetros de Cartographer-submap. Idénticos en Fase 1 y Fase 2.

A.6 Notas de alcance

- Las variantes GMapping, Hector y Cartographer son implementaciones Lua inspiradas en esos métodos y calculadas con los datos de los sensores de la escena.
- El sensor se representa con los 16 ultrasonidos del Pioneer visualizados como rayos. Las figuras de reconstrucción SLAM corresponden a esos rayos de proximidad y al arreglo ultrasónico simulado.
- phase2MappingEvidencePct mide actividad de mapeo. La IoU y la cobertura geométrica contra una referencia real quedan fuera de esa señal.
- La planificación local por punto intermedio resuelve bloqueos de una celda pequeña sin introducir una capa global adicional.
- LQR y NMPC se usan como comparaciones acotadas; el análisis central usa métodos clásicos de

seguimiento, evitación y estimación.